Empirical Software Engineering (2024) 29:150
https://doi.org/10.1007/s10664-024-10540-x

An empirical study on developers' shared conversations with ChatGPT in GitHub pull requests and issues

Huizi Hao[1] · Kazi Amit Hasan[1] · Hong Qin[1] · Marcos Macedo[1] · Yuan Tian[1] · Steven H.H. Ding[1] · Ahmed E. Hassan[2]

Accepted: 20 August 2024 / Published online: 16 September 2024
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2024

Abstract
ChatGPT has significantly impacted software development practices, providing substantial assistance to developers in various tasks, including coding, testing, and debugging. Despite its widespread adoption, the impact of ChatGPT as an assistant in collaborative coding remains largely unexplored. In this paper, we analyze a dataset of 210 and 370 developers' shared conversations with ChatGPT in GitHub pull requests (PRs) and issues. We manually examined the content of the conversations and characterized the dynamics of the sharing behavior, i.e., understanding the rationale behind the sharing, identifying the locations where the conversations were shared, and determining the roles of the developers who shared them. Our main observations are: (1) Developers seek ChatGPT's assistance across 16 types of software engineering inquiries. In both conversations shared in PRs and issues, the most frequently encountered inquiry categories include code generation, conceptual questions, how-to guides, issue resolution, and code review. (2) Developers frequently engage with ChatGPT via multi-turn conversations where each prompt can fulfill various roles, such as unveiling initial or new tasks, iterative follow-up, and prompt refinement. Multi-turn conversations account for 33.2% of the conversations shared in PRs and 36.9% in issues. (3) In collaborative coding, developers leverage shared conversations with ChatGPT to facilitate their role-specific contributions, whether as authors of PRs or issues, code reviewers, or collaborators on issues. Our work serves as the first step towards understanding the dynamics between developers and ChatGPT in collaborative software development and opens up new directions for future research on the topic.

Keywords Knowledge sharing · Conversations with ChatGPT · ChatGPT in collaborative coding · Foundation model · Issues and pull requests

Communicated by: Kelly Blincoe

Huizi Hao and Kazi Amit Hasan are both contributed equally to this work

Extended author information available on the last page of the article

1 Introduction

Recent advances in Foundation Models (FMs) hold considerable promise for automating various software engineering tasks. FM-powered tools like GitHub Copilot [1], Amazon CodeWhisperer [2], and OpenAI ChatGPT [3], are now embraced by many professional software practitioners (Zhang et al. 2023). Such technologies essentially acquire important capabilities based on massive, typically natural-language, data sets and are able to suggest recommendations to software developers, providing source code completion, automatic generation of documentation, or other types of software engineering support.

Current research on FMs for software engineering has primarily focused on evaluating the effectiveness of these models, including specialized versions modified through prompt engineering or fine-tuning, against traditional automated software engineering solutions using standard benchmarks (Jiang et al. 2023; Lu et al. 2023; Hou et al. 2023; Siddiq et al. 2023; Deng et al. 2024; Guo et al. 2024). In contrast, a limited number of studies (Vaithilingam et al. 2022; Ziegler et al. 2022; Mozannar et al. 2022; Barke et al. 2023; Liang et al. 2024) have investigated how FM-powered tools are practically employed by software developers within the software development life cycle. Most of these studies focus on GitHub Copilot, an FM-powered code generation tool via small-scale (20-42 participants) user studies or surveys.

The usage of ChatGPT in software development encompasses a broader spectrum. Unlike Copilot, which primarily aids in code completion and generation, ChatGPT is designed to generate human-like text based on the received input, i.e., *prompt*. This makes ChatGPT a more versatile tool that is applicable to a wider range of tasks beyond coding. Despite its potential, there is no research on the dynamics of developer interactions with ChatGPT, the challenges faced, and the opportunities it offers for open-source projects. Moreover, given that modern software systems are crafted by teams rather than isolated individuals, the capacity of ChatGPT to bolster and transform collaborative practices has not been explored. This potential extends well beyond its current use in individual tasks, suggesting a significant yet unexplored impact of FM-powered tools on team-based software development.

An in-depth understanding of the utilization of ChatGPT in open-source collaborative software development can provide important insights for various stakeholders. Firstly, both researchers and developers can gain a comprehensive view of the patterns and trends in how developers employ FM-powered coding assistant tools like ChatGPT to facilitate code reviews, resolve bugs, implement feature requests, and enhance collaboration within open-source projects. This analysis can uncover the strengths and shortcomings of current tools within software development workflows, and pinpoint areas for optimization. Secondly, exploring this topic allows researchers to design more realistic benchmarking tasks that effectively assess ChatGPT's practical capabilities in software engineering contexts. Lastly, in light of the emerging trend of FM-powered collaborative software development, platforms such as GitHub can benefit from these insights. For instance, they could implement new features that enable developers to more effectively reference and integrate software artifacts, like code and documentation, produced by foundation model-driven tools.

To fill the gap, we analyze developers' shared ChatGPT conversations within GitHub issues and pull requests (PRs). We postulate that these shared conversations will not only reveal how developers engage with ChatGPT in the quality assurance of open-source software

[1] https://github.com/features/copilot

[2] https://aws.amazon.com/codewhisperer

[3] https://openai.com/chatgpt

projects, such as resolving issues, but also uncover the usage of the shared conversation in collaborative coding, such as contributing to PRs and issues. Link sharing was introduced in ChatGPT in May 2023 (OpenAI 2024b), allowing users to generate a unique URL for a ChatGPT conversation. This feature enables users to share a snapshot of an entire conversation up to the point of the link being shared. Such shared conversations can help highlight important messages, reference discussions for collaborative purposes, or create a point for future reference (OpenAI 2024a). Figure 1 shows an example of a conversation with ChatGPT being shared within a code review comment on a GitHub pull request. The shared conversation contains one *turn* - a complete cycle of a developer posing a question or problem and ChatGPT responding. The code reviewer proposed a request asking the pull request author to use a JavaScript library named *Day.js* to parse and format dates. To demonstrate an example of how Day.js can be utilized, the reviewer asked ChatGPT how to replace a code snippet using Day.js, and ChatGPT responded. Then, the pull request author implemented the suggested code provided by ChatGPT. This example demonstrates the potential benefit of sharing a conversation with ChatGPT in a collaborative coding environment like GitHub.

To better understand the characteristics of shared conversations and their implications on collaborative software engineering practice, we manually analyzed a dataset containing 580 shared ChatGPT conversations within GitHub PRs and issues, specifically, 210 in PRs and 370 in issues. Our empirical study addresses the following three research questions (RQs):

RQ1: ***What types of software engineering inquiries do developers present to ChatGPT in the initial prompt?*** In this RQ, we manually examined the content of all initial prompts, 580 prompts in total, in the collected shared conversations. We developed a taxonomy composed of 16 types of software engineering-related inquiries. The most frequently encountered inquiries are about code generation, addressing conceptual questions, providing how-to guidance, assisting with issue resolution, and conducting code reviews.

RQ2: ***How do developers present their inquiries to ChatGPT in multi-turn conversations?*** In this RQ, we manually examined how developers interact with ChatGPT in *multi-turn conversations*, which consist of several rounds of prompts and responses

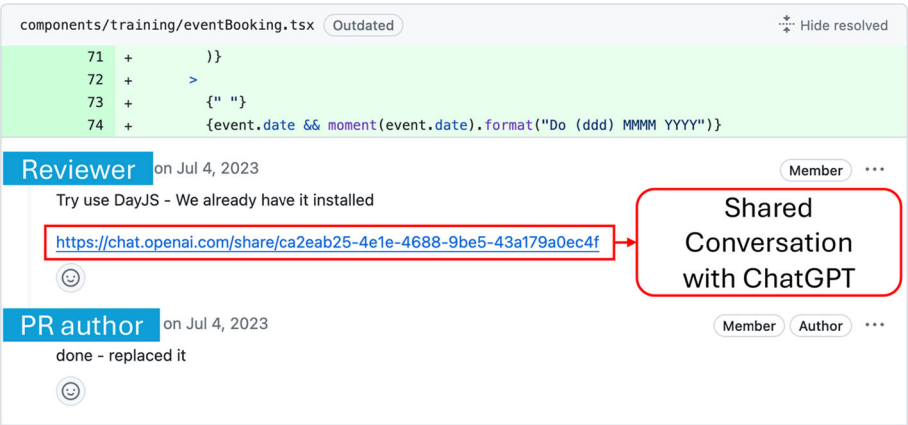


Fig. 1 An example of shared conversations with ChatGPT in a GitHub pull request. In the shared conversation, the reviewer asked "Can i replicate this functionality with DayJS? moment(event.date).format("Do (ddd) MMMM YYYY")"

between a developer and ChatGPT. Specifically, we investigated how developers structure their follow-up prompts subsequent to the initial prompt. We developed a taxonomy composed of seven types of roles a prompt plays within a multi-turn conversation. Our findings reveal that developers actively engage in multiple rounds of interactions with the aim of improving the quality of ChatGPT's responses. This is primarily achieved through the posting of follow-up questions and the refinement of prior prompts.
RQ3: ***What are the characteristics of the sharing behavior?*** In this RQ, we investigate the patterns in the sharing behaviors of developers. More specifically, we examine where those conversations are shared by whom and for what purpose. Our analysis reveals that developers utilize shared conversations as a way to complement their role-specific contributions, facilitating a more efficient and transparent collaborative process.

The main contributions of this paper are described as follows:

- We present a comprehensive analysis of how developers share their conversations with ChatGPT within the context of open-source projects, particularly focusing on GitHub pull requests and issues. Our study introduces two taxonomies to classify the dynamics of these interactions: one for the types of software engineering inquiries in developers' prompts and another for the roles of prompts in multi-turn conversations with ChatGPT. To support further research, our replication package contains 580 and 654 manually annotated prompts aligned with these taxonomies.
- Our study reveals the multifaceted usage of ChatGPT in software engineering. Beyond assisting with technical tasks, we observe the usage of shared conversations to support collaboration among developers in open-source projects.
- We provide implications for developers and software engineering researchers based on our findings. These implications offer insights to improve the use of FM-powered tools like ChatGPT further in collaborative software development.

We organize the remainder of the paper as follows. Section 2 introduces our dataset. Section 3, 4 and 5 present methodology and answers to each of the three research questions. Section 6 discusses the implications of findings for practitioners and researchers. Section 7 presents threats to validity, and Section 8 presents related work. Finally, Section 9 concludes the paper.

2 Data Collections

In this section, we introduce the used dataset for our study (Section 2.1), followed by the used method utilized in preprocessing dataset (Section 2.2) and preparing datasets for research questions (Section 2.3). Figure 2 shows an overview of the data collection process and the used data for each of our RQs.

2.1 Data Source

Our research leverages the DevGPT dataset (Xiao et al. 2024) as the primary data source. DevGPT constitutes of an extensive archive of Developer-ChatGPT interactions, featuring 16,129 prompts and ChatGPT's replies. Each shared conversation is coupled with the corresponding software development artifacts to enable the analysis of the context and implications of these developer interactions with ChatGPT. This collection was assembled by extracting shared ChatGPT links found in various GitHub components, such as source code, commits,

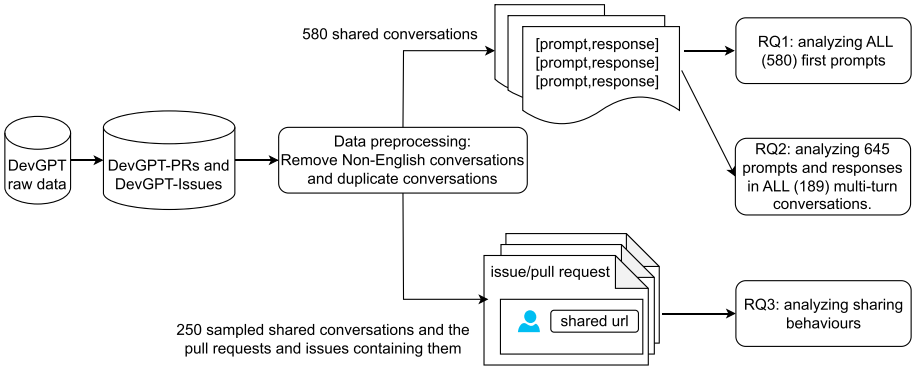


Fig. 2 Overview of the data collection process for answering three research questions

pull requests, issues, discussions, and threads on Hacker News, over the period from July 27, 2023, to October 12, 2023. The DevGPT dataset is publicly available in a GitHub repository[4], offering several snapshots. In this study, we focus on the most recent snapshot available as of October 12, 2023.[5]

2.2 Data Preprocessing

As our analysis exclusively focuses on shared conversations occurring within GitHub issues and pull requests, we only consider the corresponding data provided by DevGPT, referred to as DevGPT-PRs and DevGPT-Issues. Based on our observations, we then perform the following two data preprocessing steps:

1. The shared conversations contain sentences (prompts and replies) written in various human languages. To avoid potential misunderstandings from translating other languages different than English, we decided to only keep the conversations written in English. We utilized a Python library named *Lingua*[6] to identify conversations containing non-English content and removed those conversations. Specifically, we excluded 46 non-English conversations from DevGPT-PRs and 114 from DevGPT-Issues.
2. The shared conversations contain duplicates, i.e., conversations with identical prompts and responses. We detected duplicate conversations and kept only one instance for analysis. Specifically, we removed 20 duplicated conversations from DevGPT-PRs and 83 duplicated conversations from DevGPT-Issues.

After performing the above two data preprocessing steps, we ended up with 220 conversations from DevGPT-PRs, and 401 conversations from DevGPT-Issues.

As demonstrated in Table 1, the initial DevGPT-PRs dataset contains 254 unique pull requests, with statuses recorded as 191 merged, 36 closed, and 27 open. It is important to note that these statuses were captured at the time Xiao et al. (2024) compiled the DevGPT dataset, meaning that they may have changed since then. The DevGPT-Issues dataset originally comprised 479 unique issues, i.e., 241 closed and 238 open. Following data preprocessing, our

[4] https://github.com/NAIST-SE/DevGPT

[5] The dataset is cumulative, meaning that the latest snapshot includes all conversations shared in prior snapshots.

[6] https://github.com/pemistahl/lingua-py

Table 1 Unique PR and Issues

		Before data preprocessing	After data preprocessing
DevGPT-PRs	Merged	191	153
	Closed	36	26
	Open	27	25
	Total	254	204
DevGPT-Issues	Closed	241	185
	Open	238	186
	Total	479	371

final dataset includes 204 pull requests (153 merged, 26 closed, and 25 open) and 371 issues (185 closed and 186 open).

We observe that a single unique PR or issue may encompass multiple unique shared conversations from developers. This results in the total number of unique shared conversations in PR and issues exceeding the count of unique PRs and issues alone.

We also conclude that the duplication in the DevGPT dataset can be attributed to several factors: identical shared conversations being collected multiple times, identical conversations being shared multiple times by developers, comments containing shared conversations being cited repeatedly and collected each time they were cited, the same shared conversations being shared multiple times in one single GitHub PR or issue as well as multiple developers utilize the same prompt in their shared conversations.

2.3 Preparing Data for RQs

Figures 3 and 4 show the distribution of conversational turns within the preprocessed datasets. As shown in these figures, a large majority of shared conversations in both DevGPT-PRs (66.8%) and DevGPT-Issues (63.1%) are single-turn interactions. In the extreme case, one conversation could have more than 30 turns, i.e., 30 prompts from a developer and 30

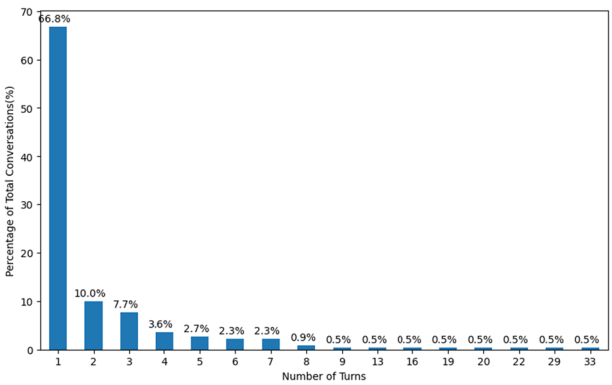


Fig. 3 Number of turns in shared conversations within DevGPT-PRs

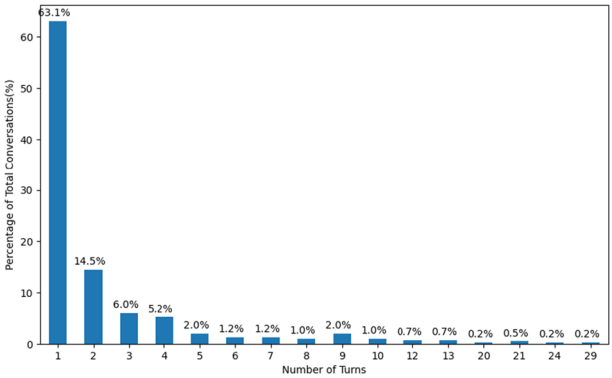


Fig. 4 Number of turns in shared conversations within DevGPT-Issues

responses from ChatGPT. Given the increased time required to label these longer multi-turn conversations and their potential to skew the analysis of prompt distribution in RQ2, we established a cutoff threshold at 8 turns, excluding all 34 conversations that exceeded this limit. Our decision was guided by two primary observations:

1. By setting the cutoff at 8 turns, we retained a significant 95% of the total conversations analyzed, i.e., 587 out of 621 conversations were kept. This level of inclusion was consistent across both DevGPT-PRs and DevGPT-issues, each showing approximately 95% of the data being selected, which ensures that our analysis remains robust and representative of the overall data.
2. Our analysis, as illustrated in Figs. 3 and 4, showed a noticeable decrease in the percentage of conversations extending beyond 8 turns. Inspired by the Elbow Method, a widely recognized technique in data analysis and machine learning, we identified this point as where the incremental benefit of including additional conversational turns sharply diminishes.

This design allows us to concentrate our investigation on the most prevalent patterns of interaction, thereby ensuring that our analysis remains closely aligned with the conversational dynamics that characterize the vast majority of the dataset. Following this cutoff, the finalized datasets encompass a total of 212 conversations for DevGPT-PRs and 375 for DevGPT-Issues.

As shown in Fig. 2, in RQ1, we analyzed the contents of 580 initial prompts [7]. In RQ2, we analyzed the content in the 645 prompts within all 189 multi-turn conversations. For RQ3, we extend our manual analysis to pull requests and issues that contain shared conversations. Specifically, we randomly sampled a statistically significant set containing 90 GitHub pull request comments and 160 GitHub issue comments containing shared conversations from DevGPT-PRs and DevGPT-Issues. The sampling is based on the results of RQ1. The detailed process is presented in the approach of RQ3 in Section 5.

[7] This number is lower than the 587 total conversations (212 from DevGPT-PRs and 375 from DevGPT-issues) due to the exclusion of seven non-English conversations that were not identified during the data preprocessing phase by the tool utilized.

3 RQ1: *What Types of Software Engineering Inquiries do Developers Present to ChatGPT in the Initial Prompt?*

Motivation The objective of RQ1 is to characterize types of SE-related inquiries that are presented by developers in their shared conversations in GitHub issues and pull requests, particularly focusing on the initial prompts of each conversation. This insight about the presented inquiries in shared conversations is crucial for several reasons. Firstly, the taxonomy provides a structured framework to categorize and understand the diverse range of software engineering inquiries developers raise to ChatGPT. Secondly, by analyzing the frequency of these inquiries, we can uncover common needs among developers when utilizing ChatGPT in collaborative software development. This insight can inform the future improvement of FM-powered tools, including ChatGPT, and the establishment of benchmarks for evaluating the effectiveness of such tools in collaborative software development. Finally, comparing inquiries across pull requests and issues allows for a comprehensive examination of the different contexts in which these inquiries are encountered.

We chose to focus on the initial prompt of each conversation in RQ1 due to two main concerns: 1) The majority of the conversations (66.8% of DevGPT-PRs and 63.1% of DevGPT-Issues) consist of single-turn interactions, wherein the initial prompt encompasses the sole task presented to ChatGPT. 2) Upon examination of multi-turn conversations, we observed that subsequent prompts never introduce new types of software engineering inquiries beyond those presented in the initial prompts.

3.1 Approach

We use open coding to manually categorize the initial prompts of all conversations in DevGPT-PRs and DevGPT-Issues. During labeling, we further removed non-English prompts that were not detected by data preprocessing. Specifically, 2 and 5 prompts from DevGPT-PRs and DevGPT-Issues were removed, respectively. Thus, our taxonomy was developed based on 210 prompts from DevGPT-PRs and 370 prompts from DevGPT-Issues. Our labeling was conducted over three rounds:

- In the first round, 50 randomly selected prompts from each dataset, i.e., DevGPT-PRs and DevGPT-Issues, are selected and categorized by five co-authors independently, i.e., 100 prompts in total. After a comprehensive discussion, the team formulated a coding book with 16 labels, excluding *others*.
- In the second round, another set of 100 prompts was individually annotated by two co-authors based on the coding book established in the first round, with an equal split sourced from both DevGPT-PRs and DevGPT-Issues. The two annotators achieved an inter-rater agreement score of 0.77, as measured by Cohen's kappa coefficient, representing a substantial agreement between the two annotators (Landis and Koch 1977). They then discussed to resolve disagreement and further refined the coding book.
- In the final round, the two annotators who participated in the second round continued and independently labeled the remaining prompts based on the coding book.

3.2 Results

We present the derived taxonomy from analyzing the 580 initial prompts in shared conversations across the two datasets in Tables 2 and 9 (in Appendix).

The results show a significant focus in the initial prompts on specific software engineering (SE) inquiries, with 198 of the initial prompts in DevGPT-PRs and 329 in DevGPT-Issues presenting such inquiries. The remainder, labeled as *Others*, comprises 12 prompts in DevGPT-PRs and 41 prompts in DevGPT-Issues, featuring either ambiguous content or information not directly related to SE inquiries, for example, "*Explain Advancing Research Communication " the role of Humanities in the Digital Era*". Note that if a developer provided background information and simultaneously asked a question in the same prompt, it won't be categorized as *information giving* because it provides more than contextual information.

In both datasets, the five most prevalent SE-related inquiries identified were *Code Generation*, *Conceptual*, *How-to*, *Issue Resolving*, and *Review*, accounting for 72% and 81% of SE-related prompts in DevGPT-PRs and DevGPT-Issues, respectively. A notable distinction between the two sources is the higher occurrence of prompts in DevGPT-PRs seeking help with *Review*, *Comprehension*, *Human language translation* and *Documentation*. Conversely, DevGPT-Issues tend to feature more requests related to *Data generation*, *Data formatting*, and *Mathematical problem resolving*.

Below, we describe each SE-related category in more detail.

(C1) Code generation This category shows the highest number of prompts in both resources, accounting for 20% in DevGPT-PRs and 27% in DevGPT-Issues, respectively. In prompts labeled as this category, developers request ChatGPT to generate code snippets based on a provided description. A follow-up analysis of these requests revealed five distinct types

Table 2 Taxonomy of the initial prompt within the shared conversations

Category	PR	Issue
SE-related:	198(100%)	329(100%)
(C1) Code generation	40 (20%)	90 (27%)
(C2) Conceptual	37 (18%)	45 (14%)
(C3) How-to	26 (13%)	74 (22%)
(C4) Issue resolving	24 (12%)	46 (14%)
(C5) Review	18 (9%)	12 (4%)
(C6) Comprehension	13 (7%)	10 (3%)
(C7) Human language translation	11 (6%)	0 (0%)
(C8) Documentation	10 (5%)	7 (2%)
(C9) Information giving	8 (4%)	8 (2%)
(C10) Data generation	4 (2%)	12 (4%)
(C11) Data formatting	2 (1%)	9 (3%)
(C12) Math problem solving	2 (1%)	9 (3%)
(C13) Verifying capability	2 (1%)	1 (0%)
(C14) Prompt engineering	1 (0%)	0 (0%)
(C15) Execution	0 (0%)	4 (1%)
(C16) Data analysis	0 (0%)	2 (1%)
Others	12	41

of requirements for code generation. As shown in Table 3, the majority of requests (48%) involve developers specifying their needs in textual form. However, it's common for developers to supply existing code, either as context for generation or as a basis for adaptation. The requirements range from code generation to more specific inquiries, such as code transformation across different programming languages or platforms and generating test code.

(C2) Conceptual In 18% of the initial prompts in DevGPT-PRs and 14% in DevGPT-Issues, developers engage ChatGPT to seek knowledge and clarification about theoretical concepts, principles, practices, tools, or high-level implementation ideas. This exploration spans questions on technical specifics, such as the memory usage limits of WebAssembly (WASM) in Chrome, to practical implementation queries, like the feasibility of developing a Redis-like cache using SQLite with time-to-live (TTL) functionality. These examples highlight ChatGPT's role not just as a direct coding assistant, but also as a conceptual guide that helps developers understand and apply complex technical knowledge they may not be familiar with.

(C3) How-to We observe that 13% of initial prompts in DevGPT-PRs and a higher percentage, 22%, in DevGPT-Issues, are from developers seeking step-by-step instructions to accomplish specific SE-related goals. For instance, "*I am doing... How can I achieve this?", and "How to make an iOS framework M1 compatible?*". Such examples illustrate ChatGPT's role in offering insights and starting points for tackling SE challenges. The higher occurrence of "How-to" prompts in DevGPT-Issues compared to DevGPT-PRs suggests that developers might turn to ChatGPT more frequently when they are in the initial stages of problem-solving, possibly lacking a concrete solution.

(C4) Issue Resolving In 12% of the initial prompts in DevGPT-PRs and 14% in DevGPT-Issues, developers seek assistance to resolve SE-related issues. Further examination of these prompts identified five types of information developers provide when seeking help with issues, as detailed in Table 4. The distribution of information types shows developers' diverse strategies to present their challenges in the prompt. The most popular method (36%), sharing error messages or traces, indicates developers' preference for direct feedback on specific errors. Combining code snippets with a detailed description of the error or unexpected behavior (19%) is more common than merely verbally describing the issues (16%). Another common type (17%) is prompts seeking assistance for troubleshooting tasks unrelated to code development, such as installing a library or setting up a system. Interestingly, developers occasionally (13%) include external links in their initial prompts, expecting ChatGPT to access these links, review the associated issues, and offer suggestions for resolution.

Table 3 Five types of code generation inquiries identified from initial prompts

Type	Freq.
Code generation based on textual specifications.	48%
Code generation with modifications to an existing implementation.	24%
Code generation incorporating code context.	12%
Code transformation across languages or platforms.	11%
Test code generation.	5%

Table 4 Five types of information in initial prompts for issue resolving assistance

Type	Freq.
Error messages, stack traces, or warnings.	36%
Code snippet with error messages or unexpected behavior description.	19%
Troubleshooting task.	17%
Description of unexpected behavior.	16%
Issue reports.	13%

(C5) Review In DevGPT-PRs, 9% of the initial prompts involve requests for ChatGPT to offer suggestions for code improvement or to compare different implementations or design choices. This frequency is notably higher than in DevGPT-Issues, where 4% of the initial prompts reflect similar requests. This distribution is expected, as developers tend to prioritize code quality and seek external validation or suggestions for enhancement in the context of PRs. Conversely, in issues, the primary focus is on identifying, discussing, and exploring potential solutions to existing problems.

(C6) Comprehension In DevGPT-PRs, 7% of the initial prompts seek ChatGPT's assistance in understanding the behavior of code snippets or software artifacts, exceeding the 3% observed in DevGPT-Issues. These prompts typically begin with phrases like "*Explain this code*". The higher prevalence in PRs suggests a focused interest in clarifying code behavior as part of the review and integration process.

(C7) Human Language Translation Notably, ChatGPT's capability in human language translation, while not specific to SE, is utilized by developers in 6% of the conversations in DevGPT-PRs, but not in DevGPT-Issues. This category encompasses requests for translating text relevant to software development from one language to another. Examples include UI components, code strings, technical documentation, and project descriptions.

(C8) Documentation Within DevGPT-PRs and DevGPT-Issues, 5% and 2% of the initial prompts from each dataset request ChatGPT for assistance with creating, reviewing, enhancing, or refining technical documentation. This category spans a wide array of documentation forms and encompasses project descriptions, issue reports, the content of pull requests, technical communications, and markdown files, among others. The requests indicate developers' recognition of ChatGPT's capability to contribute to technical documentation's clarity, accuracy, and effectiveness, reinforcing the importance of well-crafted documentation in software development.

(C9) Information Giving In DevGPT-PRs and DevGPT-Issues, 4% and 2% of initial prompts, respectively, diverge from directly requesting assistance with a specific inquiry. Instead, these prompts offer contextual information relevant to an SE-related inquiry presented in subsequent prompts. This contextual information can be a sharing of code snippets or an explanation of the developer's current project or the intended role of the ChatGPT. This approach, found in multi-turn conversations, unveils a different prompting strategy developers employ - putting background information as the initial and isolated prompt before presenting the inquiry.

(C10) Data Generation In 2% and 4% of the initial prompts in DevGPT-PRs and DevGPT-Issues, developers ask ChatGPT to create various types of data, including test inputs, search queries, icons, example of API specification, and names for websites.

(C11) Data Formatting Besides using ChatGPT to generate data, in 1% and 3% of initial prompts in DevGPT-PRs and DevGPT-Issues, developers ask ChatGPT to transform a given data into a different format. For instance, "*You are a service that translates user requests into JSON objects of type "Plan" according to the following TypeScript definitions...*".

(C12) Mathematical Problem Solving Developers turn to ChatGPT for assistance with mathematical problems in 1% of the initial prompts in DevGPT-PRs and 3% in DevGPT-Issues. These inquiries often aim to deepen one's understanding of algorithms that underpin programming tasks or to create test cases for software validation.

(C13) Verifying Capability (of ChatGPT) We encountered two and one initial prompts from DevGPT-PRs and DevGPT-Issues where one of the developers ask "*are you familiar with typedb?*". In this case, the developer assessed ChatGPT's capabilities to determine the scope of assistance it can provide for their SE-related inquiries - an establishment of trust and reliability in FM-powered tools for software development.

(C14) Prompt Engineering We find one initial prompt in DevGPT-PRs where the developer presents an initial prompt for a SE-related inquiry and asks ChatGPT to provide a better prompt. This instance presents a unique prompt engineering strategy, using ChatGPT as a tool to refine the art of prompting itself.

(C15) Execution Exclusively in DevGPT-Issues, four of the initial prompts request ChatGPT to execute a specific task, which includes running an experiment or executing code. Examples of such requests are, "*Try running that against this function and show me the result*" and "*Benchmark that for me and plot a chart*". These prompts indicate that developers believe that ChatGPT is capable of undertaking practical execution tasks, which is rarely examined in the literature.

(C16) Data Analysis Beside execution inquiries, developers also ask ChatGPT to perform data analysis tasks by providing a CSV file. We find two such cases in DevGPT-Issues. Such queries may be used to plan data preprocessing or data science projects. For instance, "*find all the entries that are present in the left and in the right column.*".

RQ1 Summary: A broad spectrum of software engineering-related inquiries, encompassing 16 distinct types, are presented to ChatGPT in the initial prompts of shared conversations in GitHub pull requests and issues. This diversity underscores ChatGPT's extensive potential in facilitating collaborative coding by matching a wide array of development needs. The inquiries mainly focus on code generation, conceptual understanding, procedural guidance ("how-to"), issue resolution, and code review.

4 RQ2: *How Do Developers Present their Inquiries to ChatGPT in Multi-turn Conversations?*

Motivation Results presented in Figs. 3 and 4 reveal that a substantial portion in DevGPT-PRs (33.2%) and DevGPT-Issues (26.9%) encompass multi-turn conversations. In single-turn conversations, developers pose a SE-related inquiry in the initial prompt and receive one response from ChatGPT, providing a clear and direct exchange. The dynamics of multi-turn conversations, however, introduce complexity. These interactions extend beyond a simple query and response, involving a series of exchanges that potentially refine, expand, or clarify the initial inquiry. This layered communication raises a question about developers' strategies to articulate their inquiries across multiple turns. Thus, we introduce RQ2, which studies the nature of developers' prompts in multi-turn conversations. To facilitate a comprehensive analysis, we further introduce two sub-RQs:

- **RQ2.1: What are the roles of developer prompts in multi-turn conversations?** This question aims to categorize the structural role of each prompt in the corresponding multi-turn conversation.
- **RQ2.2: What are the flow patterns in multi-turn conversations?** Based on the taxonomy proposed as the answer to RQ2.1, this question explores the frequent transition pattern of those identified roles of prompts in multi-turn conversations.

The answers to the above sub-RQs will provide insights for researchers about the dynamics and practices of developers in utilizing ChatGPT via multiple rounds of interactions.

4.1 Approach

In RQ2.1, we consider prompts in all 189 multi-turn conversations, i.e., 64 conversations from DevGPT-PRs and 125 from DevGPT-Issues. Following a method similar to RQ1, we used open coding to manually label 645 prompts (236 prompts from DevGPT-PRs and 409 prompts from DevGPT-Issues) in multi-turn conversations over three rounds:

- In the first round, five co-authors independently labeled randomly selected 20 conversations from both the multi-turn DevGPT-PRs and DevGPT-Issues datasets, encompassing 40 conversations and 123 prompts. Post-discussion, we developed a coding book consisting of seven distinct labels.
- In the second round, based on the existing coding book, two annotators independently labeled another set of 20 conversations from each of the multi-turn DevGPT-PRs and DevGPT-Issues datasets, a total of 144 prompts. The two annotators achieved an inter-rater agreement score of 0.89, as measured by Cohen's kappa coefficient, representing almost perfect agreement (Landis and Koch 1977). The annotators then discussed and refined the coding book.
- Finally, each of the two annotators from round two independently labeled the remaining data.

In RQ2.2, we use a Markov model (Gagniuc 2017) to analyze the conversation flow patterns by plotting a *Markov Transition Graph*. A Markov Transition Graph is a directed graph that demonstrates the probabilistic transitions between various states or nodes. In our case, each node in the graph represents a specific category developed in RQ2.1, and the

directed edges between nodes denote the likelihood of transitioning from one taxonomy to another based on the multi-turn conversations we collected. To construct this graph, we first transform each labeled multi-turn conversation into a sequence of labels. Each prompt within the conversation is labeled according to the taxonomy defined in Table 5. For instance, a conversation that contains two prompts where the first reveals the initial task and the second reveals a new task will be transferred to a sequence of "Start, Reveal the initial task, Reveal a new task, End" where "Start" and "End" indicate the beginning and the end of the conversation. After compiling all sequences, we constructed a Markov Transition Graph by calculating the transition probabilities between any two types of prompts, based on their consecutive occurrences within the sequences.

To extract meaningful insights from the Markov Transition Graph, we propose the following post-processing steps:

1. We pruned the graph by removing transitions with probabilities lower than 0.05, ensuring a focus on statistically significant relationships.
2. We refined the graph structure by removing nodes without incoming and outgoing edges, except the start and end nodes. This step ensures simplification as we only keep essential components.
3. We systematically reorganized the Markov Transition Graph into a flow chart to enhance its interpretability, offering an easier-to-understand representation of the flow patterns.

4.2 Results

4.2.1 RQ 2.1 What are the Roles of Developer Prompts in Multi-turn Conversations?

Tables 5 and 10 (in Appendix) present our proposed taxonomy to classify prompts within multi-turn conversations. Our analysis reveals that, in both pull requests and issues, multi-turn conversations contain three major types of prompts: those that pose follow-up questions (M1), those that introduce the initial task (M2), and those that are refined from a previous prompt (M3). One prompt from DevGPT-PRs and six prompts from DevGPT-Issues were categorized under "*Others*" due to their nature of being either casual conversation or lacking sufficient detail to determine their roles.

Below, we describe each category in more detail.

(M1) Iterative Follow-up In 33% and 40% of the prompts in multi-turn DevGPT-PRs and DevGPT-Issues, developers post queries that directly built upon ChatGPT's prior responses

Table 5 Taxonomy of prompts in shared multi-turn conversations

Categories	PR	Issues
(M1) Iterative follow-up	77 (33%)	163 (40%)
(M2) Reveal the initial task	62 (26%)	118 (29%)
(M3) Refine prompt	40 (17%)	56 (14%)
(M4) Information giving	18 (8%)	24 (6%)
(M5) Reveal a new task	16 (7%)	15 (4%)
(M6) Negative feedback	13 (6%)	8 (2%)
(M7) Asking for clarification	9 (4%)	19 (5%)
Others	1	6

or the ongoing context, such as debugging and repairing a solution after code generation by ChatGPT. Such iterative follow-ups typically emerge when the initial task presents a complex challenge that ChatGPT might not fully resolve in a single interaction. Consequently, developers engage in a prompt specifying a follow-up request, enabling ChatGPT to incorporate human feedback and iteratively enhance the proposed solution.

(M2) Reveal the Initial Task We find that a similar fraction, i.e., 26% in multi-turn DevGPT-PRs and 29% in multi-turn DevGPT-Issues, of prompts serve to introduce the initial task to ChatGPT. This distribution highlights that in multi-turn conversations, unlike in single-turn conversations, where the sole prompt is dedicated to outlining the primary task, there are a significant amount of prompts serving other purposes.

(M3) Refine Prompt Besides iterative follow-up (M1), developers also tend to improve the solution proposed by ChatGPT by providing a refined request prompt with additional context or constraints. The objective is to enhance the response quality for the same query posted in the previous prompt. Refined Prompts account for 17% of the prompts in multi-turn DevGPT-PRs and 14% in DevGPT-Issues.

(M4) Information Giving In 8% and 6% of the prompts in multi-turn DevGPT-PRs and DevGPT-issues, developers do not post any request for ChatGPT, but rather, share knowledge or context with ChatGPT.

(M5) Reveal a New Task We observe that 7% and 4% of the prompts in multi-turn DevGPT-PRs and DevGPT-issues are posting a new task to ChatGPT, which is distinct from the task(s) concerned in the prior prompts. This category represents a clear difference from iterative follow-ups (M1), as the new task does not relate to or build upon ChatGPT's prior responses and aims for a different goal. For example, a developer initially requested ChatGPT to generate the SQL corresponding to a Django query set and, in a subsequent prompt, asked for the SQL for a different query set, thereby shifting the focus of the conversation to an entirely new task without prior relevance.

(M6) Negative Feedback Within multi-turn conversations, a few (6% in DevGPT-PRs and 2% in DevGPT-Issues) prompts contain only negative feedback directed at ChatGPT's previous responses, without providing any information for ChatGPT to improve or further resolve. For instance, "*Your code is incorrect*", "*The same error persists*", and "*...does not work*". This category underscores instances where developers seek to inform ChatGPT of its shortcomings, without seeking further assistance or clarification.

(M7) Asking for Clarification 4% and 5% of prompts in multi-turn DevGPT-PRs and DevGPT-Issues ask ChatGPT to elaborate on its response. These requests for elaboration aim to ensure the comprehensiveness of a solution, e.g., "*Do I need to do anything else?*". They also include verification of ChatGPT's capacity to handle specific tasks, or inquiries to verify whether certain conditions have been considered in the response. Additionally, developers might ask why some alternatives were overlooked by ChatGPT, indicating a deeper engagement with the proposed solutions and a desire to understand the rationale behind ChatGPT's proposed solution.

4.2.2 RQ 2.2 What are the Flow Patterns in Multi-turn Conversations?

Figure 5 presents the resulting flow chart after applying the postprocessing steps on a Markov Transition Graph based on annotated conversations as a result of RQ2.1. The flow chart applies to multi-turn conversations in both DevGPT-PRs and DevGPT-Issues.

As illustrated in Fig. 5, multi-turn conversations typically start with the presentation of the initial task (M2) or contextual information (M4). Our detailed follow-up analysis reveals that 81% of multi-turn conversations in DevGPT-PRs and 90% in DevGPT-Issues begin by outlining the initial task. Conversely, around 13% of multi-turn conversations in DevGPT-PRs and 3% in DevGPT-Issues introduce the initial task in the second prompt. In extreme instances, the initial task is disclosed as late as the seventh turn, or, in some cases, the initial task is never explicitly presented–instead, these conversations only present information to ChatGPT without directly stating the task.

As for the complete flow, we identified the following patterns based on Fig. 5:

1. Start → reveal the initial task → iterative follow up → end
2. Start → reveal the initial task → refine prompt → (iterative follow up) → end
3. Start → reveal the initial task → reveal a new task → end
4. Start → information giving → reveal the initial task → ... → end
5. Start → reveal the initial task → asking for clarification → end
6. Start → reveal the initial task → negative feedback → end

Flow patterns (1) to (3) show the most common developer-ChatGPT interaction flows in multi-turn conversations. The initial task is disclosed in the initial prompt, followed by prompts aiming to improve ChatGPT's responses via iterative follow-up, prompt refinements, or to reveal a new task.

Pattern (4) demonstrates interaction flows started by developers providing information to ChatGPT as the first step. Then, the initial task was revealed, followed by patterns akin to (1) to (3).

Pattern (5) refers to developers asking for clarification from ChatGPT after revealing the initial task and receiving a response from ChatGPT.

Pattern (6) represents an interaction flow in which developers provide negative feedback after revealing the initial task and receiving a response from ChatGPT.

Although self-loops are excluded from Fig. 5, it's important to note that certain types of prompts, i.e., those revealing a task, refining a previous prompt, posing iterative follow-up questions, or providing information, can occur repetitively. An example of this pattern is observed when a developer successively refines their prompt across two consecutive prompts.

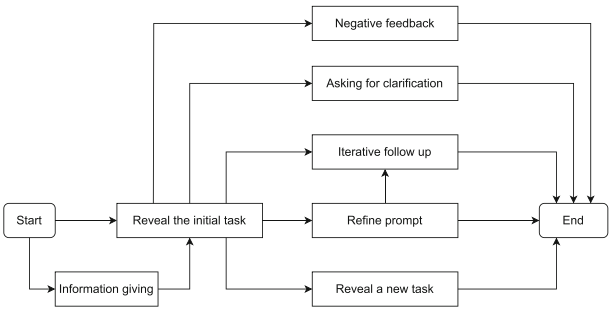


Fig. 5 A flow chart presenting common flow patterns in multi-turn conversations

RQ2 Summary: We observed similar common patterns in multi-turn conversations shared in GitHub pull requests and issues. Developers engage in iterative conversations with ChatGPT to enhance the quality of the responses. This is achieved by posting follow-up questions or refining a previous prompt to clarify the request or incorporate additional context. Most conversations introduce the initial task within the initial or second prompt, while a few begin by providing ChatGPT with relevant information. We have identified six prevalent flow patterns in multi-turn conversations, which illustrate the dynamic nature of these interactions.

5 RQ3: *What are the Characteristics of the Sharing Behavior?*

Motivation In RQ1 and RQ2, we focus on the content in shared conversations. In RQ3, we shift our attention to how and why developers share those conversations in GitHub pull requests and issues. To gain a comprehensive understanding of the characteristics associated with these sharing behaviors within the GitHub ecosystem, we propose three sub-RQs as follows:

- **RQ 3.1 Rationale: What are the rationale behind the sharing of conversations?** This question aims to uncover the underlying reasons for developers to share their conversations within GitHub PRs or issues.
- **RQ 3.2 Location: Where are the links to these shared conversations located?** This question examines where the links to shared conversations are posted in GitHub PRs and issues. GitHub PRs and issues contain different components, e.g., description, comment, and code review.
- **RQ 3.3 Person: Who shared the conversations?** This question investigates the involved developers, i.e., the roles and responsibilities of these developers within collaborative development environments like GitHub.

Answering the above sub-RQs will reveal how developers leverage these shared conversations to enhance their collaboration work within Github. This insight will, in turn, indicate areas where ChatGPT can offer more targeted support to developers in these unique contexts. By understanding these dynamics, we can identify opportunities for ChatGPT and other FM-powered software development supporting tools, to contribute to enhanced productivity and foster stronger collaboration within open-source projects.

5.1 Approach

To answer the three sub-RQs, we determine the distribution of PRs and Issues on the combined PR and Issues datasets that are the result of the data processing step. Out of 580 total shared conversations, 36.2% are from the PRs dataset, and 63.8% are from the Issues dataset. To achieve a statistically significant sample at a 95% confidence level and a margin of error of 5%, we employed stratified random sampling across the PR and Issue categories, selecting a subset of 250 conversations, i.e., 90 from PRs and 160 from Issues. Subsequently, we applied stratified sampling to each of those subsets, using the criteria outlined in Table 2. This approach ensured the distribution of software task labels in our samples remained consistent with the findings from RQ1. Then, we use the "MentionedURL" field in the

conversation profile to collect a set of URLs pointing to the PRs and issues containing the selected conversations. This process resulted in the 90 PRs and 160 issues we used for RQ3.

For RQ3.1, similar to RQ2, we use open coding to manually label the rationale behind sharing by checking the usage of the shared conversations within the context of the PRs or issues containing them over three rounds:

- In the first round, three co-authors independently labeled 30 cases each from selected PRs and issues (in total 60). Through a follow-up discussion, the annotators developed a coding book containing six distinct labels.
- In the second round, two annotators independently labeled another 20 cases from GitHub PR and GitHub Issues, a total of 40 comments, based on the coding book established in the first round. The two annotators achieved an inter-rater agreement score of 0.78. They then discussed and resolved disagreements and refined the coding book.
- In the third round, the two annotators who participated in the second round continued and equally labeled the remaining cases based on the coding book.

For RQ3.2, we manually identified the location of links to shared conversations based on the types of content provided in a PR and the issue. Specifically, for PRs, we assess whether the link is present in the PR title, description, comments, or code review comments. For issues, we determine if the link is included in the issue title, description, or comments.

For RQ3.3, we manually identified the role of the developer who shared the conversation. Specifically, we consider PR authors and code reviewers for PRs, and issue assignees, authors, and collaborators for issues.

When we labeled the collected 90 PRs and 160 issues for RQ3.1, we found cases (those labeled as *Others* in Table 6) that could not be utilized in RQ3.2 and RQ3.3. Thus, we ended up with 85 PRs and 154 issues for RQ3.2 and RQ3.3.

5.2 Results

In the following paragraphs, we first summarize the distribution of categories for each sub-RQ in Sections 5.2.1 and 5.2.2, and then present the findings of our cross sub-RQ analysis for RQ3 in Section 5.2.4.

5.2.1 RQ3.1 What are the Rationale Behind the Sharing of Conversations?

Tables 6 and 11 (in Appendix) detail a taxonomy of the derived rationale from analyzing 184 shared conversations and their corresponding PRs and issues. It identifies the top three rationales for sharing conversations in both contexts: (P1) citing the conversation as a potential solution, (P2) referring to the conversation as a source of solution, and (P3) leveraging the conversation to bolster a claim. These rationales collectively represent the majority of sharing instances, accounting for 94% in selected DevGPT-PRs and 91% in DevGPT-Issues, respectively.

Among the 250 analyzed cases, five in DevGPT-PRs and six in DevGPT-Issues were categorized as *Others* due to challenges in identifying their specific purpose. These challenges arose from several issues: 1) the removal of the link to the shared conversation, 2) the deletion of the linked pull request or issue, or 3) the content within the pull request or issue is in a language other than English. These factors contributed to the difficulty in discerning the intended use or contribution of the shared conversation in these instances.

Table 6 Taxonomy of developer's purpose of sharing

Category	PR	Issues
With clear purpose:	84 (100%)	139 (100%)
(P1) Reference to a source of solution	31 (37%)	30 (22%)
(P2) Reference to a potential solution	28 (33%)	74 (53%)
(P3) Support a claim	20 (24%)	23 (17%)
(P4) Answer a question	3 (4%)	4 (3%)
(P5) Illustrate an example	2 (2%)	8 (6%)
Direct link	1	15
Others	5	6

In one PR and 15 issues, developers only provided the URLs to the shared conversations without any explanation, forming the "*Direct link*" category.

Below, we describe each category in more detail. We use "LINKTOCHAT" to refer to the omitted links to the actual shared conversation.

(P1) Reference to a Source of Solution Developers reference shared conversations as the source of solutions in 37% of the cases within selected pull requests (with clear purpose), significantly more prevalent than in issues, where it accounts for 22%. This purpose often occurs when developers initiate a GitHub PR or issue following guidance or solutions suggested by ChatGPT. For instance, "*Current solution is based on LINKTOCHAT. Now it seems work*". This observation indicates that ChatGPT has supported developers with their contributions to open-source projects. Furthermore, it reveals that developers are inclined to include additional context and details about their contributions by referencing their interactions with ChatGPT, which will enrich the collaborative development process by providing transparent insight into the reasoning and evolution of their contributions.

(P2) Reference to a Potential Solution Beyond referencing solutions for already implemented code (P1), developers also refer to shared conversations as potential solutions. In pull requests, 33% of cases fall into this category, but it is significantly more prevalent in issues, accounting for 53%. The higher occurrence in issues is potentially due to their nature as places for exploring and debating prospective solutions aimed at issue resolution. In pull requests, we still find P2 a common category because shared conversations are leveraged to propose potential solutions for addressing code reviews, as exemplified in Fig. 1.

(P3) Support a Claim Developers shared conversations to support their claims, arguments, or suggestions in 24% of PRs and 17% of identified issues with a clear purpose. For instance, "*I have doubts about the utility of soft delete ... i consulted the chatgpt oracle, and it seems to not like soft deletion too: LINKTOCHAT*". Such instances demonstrate the strategic use of ChatGPT as an authoritative source or "oracle" to lend weight to a developer's perspective or to resolve debates within the development process.

(P4) Answer a Question In 4% and 3% of the cases in PRs and issues, developers use the shared conversation as an answer to a question raised in the previous comment. For instance, a code reviewer in a pull request asked "*This may need to be changed in transifex?...*". The pull request author answered: "*I think it does sync. LINKTOCHAT*".

Table 7 Locations of shared conversations

Location	Count (%)
In pull requests:	85 (100%)
Code review comment	30 (35%)
Pull request comment	29 (34%)
Pull request description	26 (31%)
In issues:	154 (100%)
Issue comment	97 (63%)
Issue description	56 (36%)
Issue title	1 (1%)

(P5) Illustrate an Example In a few cases, one for PR and eight for the issue, developers use the shared conversation as an example to demonstrate a concrete example of their mentioned concepts. For instance, "*We want a full exhaustive hierarchical enumeration of all the tasks a person might do on a computer. Similar to LINKTOCHAT but more exhaustive*".

5.2.2 RQ3.2 Where are the Links to these Shared Conversations Located?

Table 7 shows the distribution of locations of links to shared conversations in 85 pull requests and 154 issues. The equitable distribution of shared conversation links within pull requests indicates no particular preference for their placement. In contrast, issues demonstrate a pronounced tendency for shared conversations to be located in comments rather than in descriptions or titles.

5.2.3 RQ3.3 Who Shared the Conversations?

Table 8 shows the distribution of locations of links to shared conversations in 85 pull requests and 154 issues. The numbers reveal that: in pull requests, the likelihood of PR authors and reviewers sharing conversations is equally distributed, indicating a collaborative engagement in the sharing process. Conversely, within issues, it is predominantly the issue authors who post links to shared conversations.

Table 8 The roles of developers who shared conversations

Location	Count (%)
In pull requests:	85 (100%)
PR author	45 (53%)
Code reviewer	40 (47%)
In issues:	154 (100%)
Issue author	109 (71%)
Issue collaborator	39 (25%)
Issue assignee	6 (4%)

5.2.4 Cross sub-RQ Findings

To further understand the sharing behaviors of developers, we conducted an in-depth analysis to investigate the relationships between three considered aspects, i.e., rationale, location, and person.

Figure 6 shows two Sankey plots ("*Person-Location*" and "*Person-Rationale*" indicating the behavior patterns (where they share and why they share) of developers based on their roles in the pull requests. The width of the lines connecting these sets corresponds to the frequency of co-occurrence between the connected variables. In the *Person-Location* plot, we observe that reviewers share conversations in code review comments and pull request comments with almost equal probability, 53%, and 47%, respectively. Authors usually share their conversations in the pull request description (58%) and in the pull request comment and code review comment with similar probability, 22% and 20%, respectively. In the *Person-Rationale* plot, we observed that reviewers usually share their conversations as a reference to a potential solution (P2) (55%), then to support a claim (P3) (28%), and last as a reference to a source of solution (P1) (13%). Authors show a different sharing behavior; they mostly use shared conversations as a source of solution (P1) (58%), then support a claim (P3) (20%), and as a potential solution (P2) (13%).

Figure 7 provides a detailed analysis of shared conversations within issues, employing a *Person-Location* and a *Person-Rationale* perspective. The *Person-Location* analysis reveals that collaborators and assignees exclusively share conversations in issue comments. Issue authors (reporters) predominantly share conversations in issue comments and descriptions, with a nearly even split of 49% for comments and 51% for descriptions. One issue author

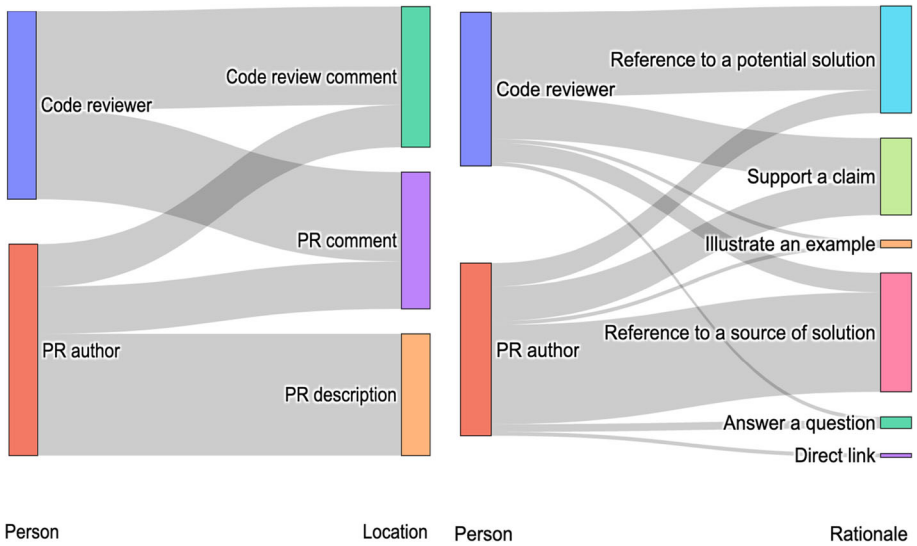


Fig. 6 The diagram presented on the left illustrates the specific location at which the individual shares the ChatGPT conversation in Github PRs. Conversely, the diagram displayed on the right delineates the rationale underlying the individual's decision to share the ChatGPT conversation in Github PR

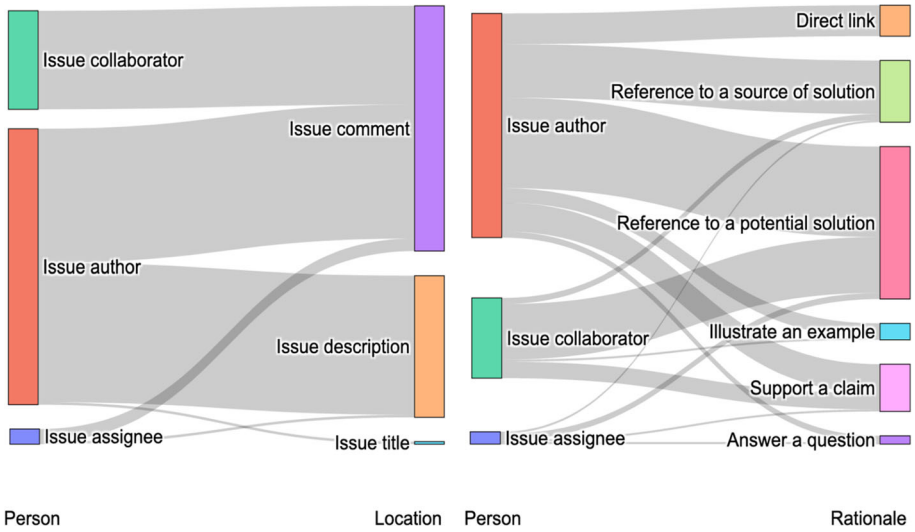


Fig. 7 The diagram on the left illustrates the location where the individual shares the ChatGPT conversation in Github issues. Conversely, the diagram displayed on the right delineates the rationale underlying the individual's decision to share the ChatGPT conversation in Github issues

even includes the link directly in the issue title. The *Person-Rationale* plot further dissects the motivations behind sharing conversations for different types of developers in issues. Issue authors primarily use shared conversations to highlight them as potential solutions (P2), counting for 41% of their shares. This is followed by references to conversations as sources of solutions (P1) and support for claims (P3), which are shared with similar probabilities of 24% and 13%, respectively. Collaborators, on the other hand, are more inclined to share conversations as a potential solution (P2) in 69% of cases and, to a lesser extent, to support a claim (P3) at 21%. Assignees prefer sharing conversations as potential solutions (P2) in 50% of instances, with the remaining equally referencing other rationales (P1, P3, and P4) at 17%, respectively.

The observed patterns of developers' sharing behaviors reflect the distinct dynamics within PRs and issues. In PRs, authors are responsible for crafting comprehensive descriptions, whereas reviewers engage primarily through code review and pull request comments. This distinction is mirrored in their motivations for sharing links to conversations: authors often share to cite the origin of their implemented solution, while reviewers tend to share to introduce or suggest alternative solutions to the author. In issues, the behavior diverges; authors actively contribute to the issue's initial description and subsequent comments. Collaborators and assignees, in contrast, limit their contributions to comments. This dynamic sees authors using shared conversations as evidence supporting their proposed solutions and substantiating their claims. Collaborators, on the other hand, frequently share conversations to present potential solutions or to bolster a claim.

Our observations underscore the role of shared conversations in facilitating collaborative problem-solving, with each participant leveraging these exchanges according to their role and the unique collaborative needs of pull requests and issues.

RQ3 Summary: Developers mainly share their conversations with ChatGPT within GitHub PRs and issues to cite these conversations as either the foundation of a solution, a prospective solution, or to support their claims. Developers leverage shared conversations to augment their role-specific inputs, effectively employing these exchanges to enhance clarity, propose alternatives, or reinforce arguments for more efficient and transparent collaborative software development.

6 Discussions

6.1 Implications

Implications for Developers Our findings suggest that developers use ChatGPT in various ways (ref. RQ2) while contributing to open-source projects, addressing different purposes (ref. RQ3) and types of tasks (ref. RQ1). These findings provide evidence and new insights into using ChatGPT and other FM-powered coding assistant tools in developers' daily workflows. For instance, tasks such as "Documentation," "Data generation," and "Data formatting" may not be commonly considered by developers, whether they have experience with FM-powered tools like ChatGPT or not. Additionally, iterative follow-ups and refining prompts remain common when using these intelligent tools. This indicates that a single round of conversation is often insufficient to complete complex practical tasks. Developers may also benefit from integrating ChatGPT more actively into their workflow, not just offline but as a means to support arguments and facilitate effective code reviews (ref. RQ3). By leveraging ChatGPT more comprehensively, developers can enhance their productivity, improve the quality of their code, and streamline collaboration within their teams.

Implications for Researchers While most recent studies on FMs for Software Engineering focus on benchmarking or enhancing them for a limited range of tasks like code generation, code translation, and program repair, our findings suggest several new directions.

First, researchers can build upon our taxonomies to further explore the dynamics of developer interactions with FM-powered tools in practice. For instance, understanding the effectiveness of conversations following different flows as identified by the taxonomy proposed in RQ2 can provide valuable insights.

Secondly, our study reveals a novel way of utilizing shared conversations with FM-powered tools in collaborative software development. Researchers could investigate the short-term and long-term impact of such behaviors on developer productivity and code quality. For example, one could analyze if sharing conversations with ChatGPT improves the efficiency and quality of code reviews.

Thirdly, the findings from RQ2 highlight the frequent use of multi-turn strategies to improve ChatGPT'solutions iteratively. This finding motivates future investigations into the efficiency of developers' prompting techniques within these multi-turn conversations. Specifically, whether developers have applied the best practices in prompt engineering in practice and whether improved prompts can effectively alter the flow of these interactions is a future direction for enhancing the utility and effectiveness of FM-powered tools in software development.

Lastly, our findings from RQ1 highlight the need for future benchmark designs to evaluate the impact of FMs on various software engineering tasks. In RQ1, we identified multiple

types of input for code generation and issue resolution inquiries that are not fully captured by existing benchmarks. For instance, the widely recognized code generation benchmark, HumanEval (Chen et al. 2021), relies on textual specifications and method signatures. However, our analysis shows that nearly half of the code generation prompts (47%) include initial code drafts alongside textual descriptions. Similarly, our examination of prompts categorized under "Issue resolving" indicates that a significant portion (36%) of issue resolution requests involve sharing error messages or execution traces, often without accompanying source code. We also observed many other tasks beyond code generation and issue resolution, such as code review, conceptual questions, and documentation, which are rarely considered as benchmark tasks for FM-powered software development tools. Addressing these tasks in future research could broaden the scope and applicability of FMs in software engineering.

Implications for Software Project Hosting Sites Our findings offer several insights for software project hosting sites like GitHub and GitLab, highlighting opportunities to enhance their platforms to better support developers' interactions with FM-powered tools like ChatGPT. For instance, hosting sites can facilitate seamless integration with FM-powered tools to streamline various development tasks. For example, integrating ChatGPT directly into pull requests and issues can help developers quickly generate code snippets, refine prompts, and resolve issues without leaving the platform. This integration can enhance productivity and collaboration.

Sharing conversations with ChatGPT via links in PRs and issues can help track the influence of FM-powered tools on the evolution of a software project. However, our observation of instances where shared URLs became inaccessible, possibly due to the removal of conversations from the user's ChatGPT interface. This observation suggests the need for better tracking mechanisms. Hosting sites could provide functionalities to store and reference these discussions and code generated with the support of FM-powered tools within the platform, ensuring they remain accessible and traceable. This would help software project teams monitor the impact of utilizing FM-powered tools in their collaborative development workflow.

6.2 Comparing our Taxonomy with Existing Literature

Our taxonomy proposed for RQ1 on the types of inquiries developers asked in their shared conversations with ChatGPT was inspired by taxonomies from the literature, i.e., the seven categories proposed by Beyer et al. (2020) and the 13 categories proposed by Hata et al. (2022). However, rather than directly adopting these taxonomies, we applied open coding to systematically label the conversations between developers and ChatGPT, tailoring the existing frameworks to fit our specific context. Detailed differences are summarized in Table 12 (in Appendix).

The primary rationale for this design choice stems from our observation that interactions between developers and ChatGPT differ from those on Stack Overflow and GitHub. For instance, in Beyer et al. (2020), 30.1% of the labeled questions pertain to "API usage" and "API change." However, our dataset did not contain many inquiries centering around API usage or changes. Instead, we observed more questions related to code generation and how-to tasks. As a result, we introduced these categories to better reflect the nature of the inquiries in our context, rather than retaining the API-related categories.

It is common practice to adapt categories when switching contexts, even when focusing on types of developers' questions. For example, the two API-related categories present in Beyer et al. (2020)'s taxonomy do not appear in Hata et al. (2022)'s taxonomy. Furthermore, our dataset includes new types of questions such as "comprehension," "prompt engineering," and "documentation" (where documentation inquiries are about modifying documentation rather than for learning purposes). These types of questions are not covered in the previous taxonomies, likely due to the different nature of interactions on Stack Overflow and GitHub or the new capabilities of FM-powered tools like ChatGPT. For instance, it is uncommon for developers to ask code comprehension questions or help for improving documentation on Stack Overflow, as they are not specific technical problems with code. By creating a new taxonomy tailored to the specific context of ChatGPT-developer interactions, we aim to contribute a new perspective to the research field. Moreover, this approach allows us to more accurately reflect and analyze the unique aspects of such conversations.

Our RQ2 taxonomy was inspired by existing frameworks proposed by Huang et al. (2018) and Qu et al. (2019). For instance, they also have same or similar categories like "Information Giving", "Follow Up Questions", "Negative Feedback", and "Original Question". However, as with our findings in RQ1, we discovered that these taxonomies could not be directly applied to capture the unique patterns in multi-turn conversations between developers and ChatGPT. For instance, the category "Refine prompt" is prevalent in our dataset but is not identified in the previous taxonomies. Additionally, since previous studies focused on human-human conversation labeling, they included categories like "Solution Proposal'. In contrast, we only label the prompts proposed by developers when communicating with ChatGPT. Therefore, we applied open coding to systematically label all multi-turn conversations, ensuring our taxonomy accurately reflects the distinct characteristics of these interactions.

7 Threats to Validity

Internal validity

- **Open Coding**: Open coding was employed in our study as the major methodology we conducted. This approach introduces subjectiveness to our results as the annotators may hold different understandings of the coding schema. To mitigate this threat, we follow the best practices as described below: (1) we conduct multiple rounds of individual coding, and we ensure the Cohen's Kappa score reflecting the agreement between the annotators is at least substantial; (2) all disagreements are discussed with all annotators until agreement reached; (3) two rounds of revisit on the entire label-set are performed after individual labeling to mitigate any miscoded.
- **DevGPT Dataset Quality**: We performed random manual checks on the DevGPT dataset by examining the shared conversation content collected in the dataset against the original conversations. We found that the DevGPT data misses the last turn of conversation in multi-turn conversations. As a result, the distribution of conversation turns may be different in real data than what is presented in this paper. However, our study primarily focuses on the categorization of software engineering inquiries, the role of prompts and the characteristics of developers' sharing behaviors. Therefore, the missing data points do not impact our results.

- **Performance of *Lingua***: The Python library *Lingua*[8] was utilized to remove conversations containing non-English content during the data preprocessing step. Out of the 724 conversations identified as English by Lingua, we discovered through manual labeling in RQ1 that seven were non-English. These seven conversations were excluded from the analysis in RQ2, ensuring that only English conversations were included. Lingua demonstrated an overall precision of 99% and a recall of 92% in identifying English conversations.
- **Cutoff at 8 Conversational Turns for Data Preparation**: In our study, we implemented a cutoff at 8 conversational turns to manage the scope of data analysis in Section 2.3, particularly focusing on the bulk of interactions that conform to this limit. While this decision was informed by the observation of diminishing returns in additional data beyond this point, as identified using the Elbow Method, it introduces a potential internal threat to the validity of our findings. To mitigate this threat, we further examined the 34 removed long multi-turn conversations and found no evidence that their inclusion would alter the findings of RQ1-RQ3.
- **Mixed Content Prompts**: In RQ1 and RQ2, one prompt may be related to multiple categories that we identified, but we only kept one category per prompt. During our manual labeling, we observed that most prompts contain one clear inquiry (with one specific goal) from developers. This means we can classify them into one category. Only one prompt in our manual labeling process contains more than one inquiry, and we classified it according to the main inquiry. When a prompt includes contextual information and a specific inquiry within the same prompt, we classify it following the specific inquiry of the prompt, but not *Information Giving*.

External Validity

- **DevGPT Dataset** Our results are derived from the specific dataset, DevGPT, where only developers shared conversations in GitHub PRs and Issues are considered. Furthermore, the dataset only contains data collected up to October 2023. Thus, our results may not generalize to all Developer-ChatGPT conversations. Developers shared conversations and usage of these shared conversations existing in other open-source platforms might vary. Developers shared conversations and usage in general contexts, i.e., when these shared conversations are not posted in GitHub PRs, and issues may not follow the findings discussed in this paper. We encourage researchers to reproduce our study on a larger dataset or data collected from other open-source communities.
- **Duplicate Data in the DevGPT Dataset** In the data preprocessing step (ref. Section 2.2), we only kept one copy of the duplicate conversations, which are conversations with identical prompts and responses. We then performed an in-depth analysis regarding all detected duplicated sets and investigated the reasons behind the presence of duplicate data in the DevGPT dataset. We identified several causes:

  1. Identical shared conversations were collected multiple times, resulting in duplicate data in the DevGPT dataset. For example, shared conversation https://chat.openai.com/share/7faf4574-49a3-4e3b-9bd0-016f62f864c9 in GitHub issue https://github.com/mccaffary/GPT-4-ChatGPT-Project-Euler/issues/1#issuecomment-1715655206 were collected multiples times. Possibly due to the reason that one issue comment

[8] https://github.com/pemistahl/lingua-py

contains more than one shared conversation, and Xiao et al. (2024) collected the comment multiple times while processing each shared conversation.

2. Developers shared the conversations with ChatGPT in multiple PRs and issues, and all occurrences of the shared conversations were collected. For example, conversation https://chat.openai.com/share/245724e7-0a3c-4001-9fef-fb72c4a4acc6 were shared in three PRs and within the same context: https://github.com/DogeisCut/extensions/pull/2, https://github.com/DogeisCut/extensions/pull/1 and https://github.com/TurboWarp/extensions/pull/1035 to demonstrate the hallucination of ChatGPT.
3. Developers cited previous comments containing shared conversations in follow-up comments, and DevGPT collected the shared conversations each time they were cited. For example, conversation https://chat.openai.com/share/67f598d7-89c2-491a-9a7b-a027120e6acf was first shared in the issue comment https://github.com/mdn/content/pull/27070#issuecomment-1570821781, then cited in a following issue comment https://github.com/mdn/content/pull/27070#issuecomment-1571207236.
4. Developers shared the same conversation multiple times in one single PR/issue and all occurrences of the conversation are collected in DevGPT. For example, conversation https://chat.openai.com/share/c2ce8b0a-5fa3-4871-ae5a-7fb46a16ce6a was shared multiple times by the developer in GitHub issue https://github.com/jabrena/spring-boot-user-beans/issues/36.
5. In multiple shared conversations, different developers used the same prompt and asked ChatGPT the same question. For example, the same prompt was used by two developers in two shared conversations https://chat.openai.com/share/b4d90deb-49fd-4c95-982f-dedb1c9c6e33 and https://chat.openai.com/share/e8f7eb03-807f-44f3-b7fe-3d883dcf08e4.

We believe that keeping one copy from each duplicate conversation set does not affect our study results, as they do not alter the categorization of software engineering inquiries, the role of prompts, or the characteristics of developers' sharing behaviors.

Construct Validity To ensure our taxonomy presented in this study best captures the data, we develop our coding book based on previous studies and expand on them to include ChatGPT-specific scenarios. We build our coding book for RQ1 based on the taxonomy proposed by Beyer et al. (2020) and Hata et al. (2022). Our RQ2 coding book is expanded on the taxonomy proposed by Huang et al. (2018) and Qu et al. (2019). Details on the comparisons of our taxonomy and existing ones in the literature are discussed in Section 6.2.

8 Related Work

In this section, we first present related work on the usability of FM-powered tools (Section 8.1). As our study analyzes shared conversations in GitHub issues and PRs, we also present existing studies on analyzing conversations related to software engineering (Section 8.2) and link sharing in SE platforms, i.e., GitHub and Stack Overflow (Section 8.3).

8.1 Understanding Developers' Interactions with FM-powered Software Development Tools

Several studies investigated the interactions between FM-powered software development tools and developers. These studies adopted either user studies or surveys as their methodology, with a significant emphasis on GitHub Copilot, a code completion tool powered by OpenAI's Codex model.

User studies, such as those conducted by Vaithilingam et al. (2022); Mozannar et al. (2022); Barke et al. (2023) and Ross et al. (2023), were performed to understand how developers interact with GitHub Copilot. Vaithilingam et al. (2022) reported a general preference among developers for incorporating Copilot into their daily programming tasks via a user study with 24 participants. However, they also found that developers face challenges in understanding and debugging the code that Copilot generates. Mozannar et al. (2022) developed a taxonomy of 12 programmer activities associated with GitHub Copilot use, such as verifying suggestions and debugging. Their analysis of 21 developers' interactions with Copilot revealed that a substantial portion of coding sessions were spent on activities like double-checking and editing Copilot's suggestions. Barke et al.'s user study 2023 with 20 participants reported a bimodal usage pattern among developers: they employ Copilot both to speed up familiar tasks ("acceleration mode") and to explore solutions for new challenges ("exploration mode"). Moreover, programmers may defer thoughts on suggestions for later and simply accept them when displayed. Ross et al. (2023) conducted a user study with 42 developers with follow-up surveys using their proposed program assistant, which relies on OpenAI's codex API to support code edit suggest and chat function. They found that 83% of participants rated the importance of the ability to ask follow-up questions as being "somewhat" or "a great deal".

On the survey front, large-scale studies by Ziegler et al. (2022) and Liang et al. (2024) explored developer perceptions of productivity and usability challenges with AI programming assistants, including GitHub Copilot. Ziegler et al. (researchers from GitHub) collected survey responses from 2,631 developers with IDE usage data. They identified the acceptance rate of Copilot's suggestions as a significant predictor of perceived productivity. Interestingly, acceptance rates varied across programming languages, i.e., 23.3%, 27.9%, and 28.8% of Copilot's suggestions were accepted for TypeScript, JavaScript, and Python, respectively, and 22.2% for all other languages. More recently, Liang et al. (2024) performed another survey with 410 developers focusing on the usability challenges of many AI programming assistants, including GitHub Copilot and ChatGPT. Their results show developers are most motivated to use AI programming assistants because they help developers reduce keystrokes, finish programming tasks quickly, and recall syntax. They also found that the most frequent usability challenges were understanding how input leads to output code and how to control the tool's generations.

Similar to prior studies, our study aims to explore the potentials and limitations of FM-powered tools, i.e., ChatGPT, in software development by analyzing the interaction between developers and ChatGPT. Different from them, we explore ChatGPT's role in supporting developers' collaboration in coding on GitHub by analyzing developers' shared conversations with ChatGPT on GitHub issues and PRs. We deployed a mixed research method, including qualitative and quantitative studies, rather than using user studies or user surveys.

8.2 Conversation Analysis in Software Engineering

In software development, conversations play a crucial role in facilitating collaboration, knowledge sharing, and problem-solving among developers. These interactions vary widely, ranging from email lists to issue discussions on open-source software (OSS) platforms like GitHub to technical queries on community forums such as Stack Overflow. Each type of conversation has unique characteristics and serves distinct purposes within the software development lifecycle.

Di Sorbo et al. (2015) manually annotated 100 emails taken from the Qt project development mailing list and proposed a taxonomy that categorized communications between developers into six types, including feature request, opinion asking, problem discovery, solution proposal, information seeking, and information giving. Building upon this framework, Huang et al. (2018) sampled 5,408 sentences from comments recorded in issue tracking systems of four large and popular projects hosted on GitHub. Their manual categorization refined Di Sorbo et al.'s taxonomy by adding new categories, such as "aspect evaluation" and "meaningless (less informative)". Besides general categorization, some studies focus on specific topics from conversations. Arya et al. (2019) also investigated issue discussions, but with a focus on fine-grained information types mentioned in conversations. They identified 16 distinct types of information and compiled a labeled corpus of 4,656 sentences. Nurwidyantoro et al. (2022) annotated 1,097 issue discussions collected from three Android projects to understand human values in software development artifacts. Their results show that value themes could be found in issue discussions (33% of the inspected issues). Beyond issues, studies have also explored conversations in pull requests and online chat rooms. Viviani et al. (2019) proposed an automated solution to locate the points of the discussion where developers discuss design in pull requests. Shi et al. (2021) conducted a comprehensive empirical study on developers' live chat in gitter, exploring interaction dynamics, community structures, discussion topics, and interaction patterns. In contrast to existing research that primarily focuses on developer-to-developer communication, our study focuses on developer-to-tool interactions. We introduce two new taxonomies: one that categorizes SE-related inquiry types and another that characterizes the role of sentences within multi-turn conversations, drawing inspiration from literature for certain categories like "information giving".

Our findings in RQ1 highlight that most shared conversations with ChatGPT in GitHub issues and pull requests revolve around seeking assistance with SE tasks, i.e., these conversations are predominantly information-seeking conversations. Before the advent of FM-powered tools in software development, such inquiries were commonly posed on Q&A sites like Stack Overflow (SO). In the literature, analyzing questions developers post on SO is a common practice to understand developers' faced challenges in a specific domain, such as mobile development (Rosen and Shihab 2016), blockchain (Wan et al. 2019), deploying deep learning application (Zhang et al. 2019), and all kinds of domains (Barua et al. 2014). However, the studies most pertinent to our research are those that focus on categorizing the general types of questions posed by developers on SO rather than their specific technical content. The primary distinction between these two categorization strategies is that the latter approach groups questions into broader categories, such as how-to, thus focusing on the underlying motivations and types of information developers seek. Yet, the studies most relevant to our research are those aimed at categorizing the general types of questions posed

by developers on SO. Treude et al. (2011) were the first ones investigating the question categories of posts of SO. In 385 manually analyzed posts, they found 10 question categories: how-to, discrepancy, environment, error, decision help, conceptual, review, non-functional, novice, and noise. Similarly, Rosen and Shihab (2016) manually categorized 384 posts of SO for the mobile operating systems Android, Apple, and Windows, each into three main question categories: How, What, and Why. More recently, Beyer et al. (2020) have advanced this line of work by manually classifying 1,000 SO questions, proposing a new taxonomy that amalgamates all previously identified question categories. This taxonomy comprises seven high-level categories: API usage, conceptual issues, discrepancies, errors, reviews, API changes, and learning. Our taxonomy in RQ1, inspired by Beyer et al., aligns with many SE-related inquiries observed on SO but also distinguishes the unique types of inquiries directed at ChatGPT, such as those related to documentation improvement, code comprehension, data generation, and data formatting. Moreover, we discuss the role of sentences (developers' prompts) in multi-turn conversation and developers' sharing behavior, which is not covered in prior studies.

8.3 Linking Sharing in GitHub and Stack Overflow

Link sharing serves as a pivotal method for knowledge sharing within developer communities, particularly on Q&A sites like Stack Overflow (SO) and social coding platforms such as GitHub. This practice, widely recognized for facilitating the exchange of insights about software development tools and libraries, underpins much of the collaborative ethos in these communities.

There has been extensive research on developers' link-sharing behaviors on Stack Overflow. Gómez et al. (2013) observed that a considerable portion of the shared links on SO are aimed at spreading knowledge about new software development tools and libraries. Ye et al. (2017) further explored this phenomenon, examining the structure and dynamics of SO's knowledge network through link sharing. Their findings highlight that developers share links for a variety of reasons, with referencing information for problem-solving being the most common purpose. Baltes et al. (2020) provided an in-depth analysis of how and why documentation links are cited within SO posts by examining 759 shared links to understand their roles and the importance of context in interpreting these references. More recent studies by Liu et al. (2021, 2022) have analyzed the characteristics of broken shared links and the patterns of repeatedly referenced links within SO posts.

Beyond Stack Overflow, research has extended to the use of links within GitHub's ecosystem, particularly in issues and pull requests. Zampetti et al. (2017) explored the extent and purposes behind developers' references to external online resources in pull requests, indicating a strong inclination towards acquiring new knowledge or addressing specific problems. Zhang et al. (2018) observed that developers tend to link more cross-project or cross-ecosystem issues over time. Li et al. (2018) conducted a comprehensive study to understand why developers create links within issues and pull requests on GitHub and how these links impact software development. They manually identified six types of relationships in linking behavior: dependent, duplicate, relevant, referenced, fixed, and enhanced. Hata et al. (2019) collected 9.6 million links in source code comments. They found that over 80% repositories contain at least one link, pointing to the prevalence of link sharing for purposes such as

providing metadata or attributions. Wang et al. (2021) emphasized the critical role of shared links in code review, demonstrating how they serve as vital resources for both authors and review teams. Xiao et al. (2023) highlighted the issue of link decay in commit messages and the various purposes for which developers include links, primarily for context provision.

Our results in RQ3 align with findings from prior studies. For instance, similar to these investigations, we identified instances where shared links, specifically those encapsulating dialogues between developers and ChatGPT, were broken. In response to this issue, we observed a practice among some developers who, instead of solely sharing a link, opted to embed the entire conversation within issue comments or pull requests as a form of reference (when manually analyzing PRs and issues in RQ3). We also found the shared links play an important role in supporting collaboration in coding. Unlike the predominant trend observed in SO, where documentation links are frequently cited, or in GitHub, where cross-project and ecosystem software artifact links prevail, our study sheds light on the unique nature of links to ChatGPT conversations. These links represent a novel vector for knowledge sharing and collaboration.

9 Conclusion and Future Work

In this paper, we study the role of ChatGPT in collaborative coding by analyzing developers' shared conversations with ChatGPT in GitHub pull requests and issues, leveraging the DevGPT dataset.

Our key findings include: (1) Developers seek ChatGPT's assistance across 16 types of software engineering inquiries. The most frequently encountered requests involve code generation, conceptual understanding, how-to guidance, issue resolution, and code review. (2) In code generation and issue resolution tasks, developers often go beyond the conventional inputs of textual descriptions or buggy code, which are standard benchmarks for FMs in code generation and program repair. This indicates a broader range of inputs being utilized in real-world scenarios. (3) Developers engage with ChatGPT during multi-turn conversations through iterative follow-up questions, prompt refinement, and clarification inquiries. These methods are employed to enhance the quality and relevance of ChatGPT's responses progressively. (4) Developers with different roles–such as issue authors, PR authors, and code reviewers–utilize shared conversations with ChatGPT to supplement their role-specific contributions. This practice aims to improve the efficiency and transparency of collaborative software development processes.

In the future, we plan to propose automated approaches that can automatically identify the types of prompts based on our taxonomies. We also plan to explore whether the existing best practices for prompt engineering have been applied in the collected shared conversations and if applying them will influence the flow of multi-turn conversations.

Appendix

A RQ1 Taxonomy and Examples

Table 9 Detailed description of the 16 SE-related inquiries observed from the initial prompt within the shared conversations

Category	Description	Examples
(C1) Code generation	Developers request code snippets generated based on a provided description.	"Create a python script to send a DNS packet using scapy with a secret payload".[a]
(C2) Conceptual	Developers seek knowledge and clarification about theoretical concepts, principles, practices, tools, or high-level implementation ideas.	"is it bad practice to use v-html in vue?".[b]
(C3) How-to	Developers seek step-by-step instructions to accomplish specific SE-related goals.	"I am writing a nextjs app. I want to run a simple function periodically. How can I achieve this".[c]
(C4) Issue resolving	Developers seek assistance to resolve SE-related issues.	"How do I fix this python error: No module named 'bs4".[d]
(C5) Review	Developers request suggestions for code improvement or to compare different implementations or design choices.	"Any suggestions on how I might optimize this code. The processing time seems a bit slow: ...".[e]
(C6) Comprehension	Developers seek assistance in understanding the behavior of code snippets or software artifacts.	"what does this do? ...".[f]
(C7) Human language translation	Developers request human language translation for translation tasks.	"convert to french ...".[g]
(C8) Documentation	Developers request assistance with creating, reviewing, enhancing, or refining technical documentation.	"how can the following documentation be improved ...".[h]
(C9) Information giving	Developers offer contextual information relevant to an SE-related inquiry presented in subsequent prompts.	"You are an R and SQL expert".[i]
(C10) Data generation	Developers request to create various types of data, including test inputs, search queries, icons, examples of API specifications, and website names.	"I want to add an option to my CLI tool for importing CSV files into a database - the option will mean "if you see an empty string, store a null" - give me lots of options for that name, each with a short justification".[j]
(C11) Data formatting	Developers request to transform a given data into a different format.	"can you reorder the columns below to be | What | Description | When | Status | ...".[k]
(C12) Math problem solving	Developers seek assistance with mathematical problems.	"By starting at the top of the triangle below and moving to adjacent numbers on the row below, the maximum total from top to bottom is 23. ... Find the maximum total from top to bottom of the triangle below: ...".[l]

Table 9 continued

Category	Description	Examples
(C13) Verifying capability	Developers assess capabilities to determine the scope of assistance that can be provided for their SE-related inquiries.	"are you familiar with typedb?".[m]
(C14) Prompt engineering	Developers present an initial prompt for a SE-related inquiry and asks for a better prompt.	"I am telling an LLM about the "arguments" property of an object. The arguments property must be of type 'string'. My description of the arguments property is "The arguments to pass into the script being executed". How can I concisely and effectively modify the description to inform the LLM that the arguments must be in json format?".[n]
(C15) Execution	Developers request execution on a specific task, for instance, running an experiment or executing code.	"I wrote this code: ... Try running that against this function and show me the result: ...".[o]
(C16) Data analysis	Developers ask ChatGPT to perform data analysis tasks by providing a CSV file.	"xy_HOLISTIC_OPENSIM .csv Spreadsheet I'm hoping to do some EDA of the above data".[p]
Others	Ambiguous content or information not directly related to SE inquiries.	"Write a poem about sharing talks with AI".[q]

[a] https://chat.openai.com/share/720c1c30-6ee6-444b-b3b4-48eaec04bcd4
[b] https://chat.openai.com/share/863a44da-3728-4fe5-83b8-285f4912abeb
[c] https://chat.openai.com/share/2a82e086-024e-4b63-968d-7c8834c9642c
[d] https://chatgpt.com/share/53a0b68f-b8da-4b21-baef-e5f28b0b4270
[e] https://chatgpt.com/share/89d2b9cc-8a2d-42d7-82f5-64c55683022e
[f] https://chatgpt.com/share/b50316a4-0f63-4813-8888-9cb3ca68b7f1
[g] https://chatgpt.com/share/be516fdf-e0e6-46eb-b2b9-21a35147cc06
[h] https://chatgpt.com/share/065c4563-3e52-4520-a8f2-c9b494023f4e
[i] https://chatgpt.com/share/148d8cd0-2784-4659-9d4a-cd19dbf340c1
[j] https://chat.openai.com/share/c947b738-ee7d-419c-af90-bc84e90987da
[k] https://chat.openai.com/share/e93e3816-c8ad-4276-8f95-a8a0a1ba57b4
[l] https://chatgpt.com/share/a8e37b34-15f4-4b59-81b5-ea50365c3265
[m] https://chatgpt.com/share/60645554-8beb-45d3-b996-0174b1c41c06
[n] https://chatgpt.com/share/e1f297bd-e6fb-491a-97f5-7754e2f3ca12
[o] https://chat.openai.com/share/b9873d04-5978-489f-8c6b-4b948db7724d
[p] https://chat.openai.com/share/50c8e7f0-eec8-42a7-9289-188e8b1251fe
[q] https://chatgpt.com/share/401d72e6-ce08-4c22-9cbf-edf237b275ad

B RQ2 Taxonomy and Examples

Table 10 Detailed description of the 7 roles of prompts observed from the multi-turn conversations within the shared conversations

Category	Description	Examples
(M1) Iterative follow-up	Developers post queries that are directly built upon ChatGPT's prior responses or the ongoing context, such as debugging and repairing a solution after code generation by ChatGPT.	In the second prompt, the developer asked ChatGPT to create mock input and test the function ChatGPT generated in the previous response. This prompt was built on the developers' previous prompt that asked ChatGPT to review code.[a]
(M2) Reveal the initial task	Prompts serve to introduce the initial task to ChatGPT.	The developer's initial prompt asked ChatGPT to review two pieces of code. This code review task is the initial task that the developer presented to ChatGPT in the whole shared conversation.[b]
(M3) Refine prompt	Developers tend to improve the solution proposed by ChatGPT by providing a refined request prompt with additional context or constraints.	In the third prompt, the developer requested ChatGPT to generate more advanced test input to improve the test case ChatGPT generated in the previous response.[c]
(M4) Information giving	Developers do not post any request for ChatGPT, but rather, share knowledge or context with ChatGPT.	In the initial prompt, the developer only shared context with ChatGPT.[d]
(M5) Reveal a new task	Prompts post a new task to ChatGPT, which is distinct from the task(s) concerned in the prior prompts.	In the second prompt, the developer requested ChatGPT's assistance on a Nova File Manager problem, which is a new task compared to the symlink problem she/he posted about in the first prompt.[e]
(M6) Negative feedback	Prompts contain only negative feedback directed at ChatGPT's previous responses, without providing any information for ChatGPT to improve or further resolve.	The developer only provided negative feedback to ChatGPT in the third prompt of the shared conversation.[f]
(M7) Asking for clarification	Prompts that ask ChatGPT to elaborate on its response.	The developer asked ChatGPT to clarify the provided solution.[g]
Others	Prompts' nature of being either casual conversation or lacking sufficient detail to determine their roles.	The developer did causal chat with ChatGPT in the first prompt.[h]

[a] https://chat.openai.com/share/acd17c82-87f6-499a-9d09-0b675b4ef4ea
[b] https://chat.openai.com/share/acd17c82-87f6-499a-9d09-0b675b4ef4ea
[c] https://chat.openai.com/share/acd17c82-87f6-499a-9d09-0b675b4ef4ea
[d] https://chatgpt.com/share/478c9404-79d8-4632-b82a-bb9fbad9648a
[e] https://chatgpt.com/share/71395f9c-3cc5-4de3-b5b8-524bc2cb6b7d
[f] https://chatgpt.com/share/f612708b-692b-487f-861b-7081f8bec5c5
[g] https://chatgpt.com/share/86111521-7f64-4c67-8273-43a21d76c8df
[h] https://chatgpt.com/share/f2b7484c-8455-47e2-bb45-1725138dcdb0

C RQ3 Taxonomy and Examples

Table 11 Detailed description of the developers' sharing behaviors observed from the GitHub PRs and issues containing the shared conversations

Category	Description	Examples
With clear purpose		
(P1) Reference to a source of solution	Developers reference the shared conversation as the source of a solution they implemented in a PR or Issue.	The author of the PR explained that the currently implemented solution was based on a ChatGPT response as demonstrated in the shared conversation.[a]
(P2) Reference to a potential solution	Shared conversations are leveraged to propose potential solutions for addressing code reviews or issues.	The coder reviewer suggested a potential solution to an issue she/he saw during code review.[b]
(P3) Support a claim	Developers share conversations to support their claims, arguments, or suggestions.	The code reviewer doubted using "soft delete" due to potential issues resulting from it. He/she consulted ChatGPT, and ChatGPT did not recommend using soft delete either. The code reviewer was using ChatGPT response to support his/her claim on avoiding using soft delete.[c]
(P4) Answer a question	Developers use the shared conversation as an answer to a question raised in the previous PR or issue comment.	In the previous code comment, a reviewer asked a question on synchronization of transifex and in the following comment, the PR author used ChatGPT's response to answer the question.[d]
(P5) Illustrate an example	Developers use the shared conversation as an example to demonstrate a concrete example of their mentioned concepts.	In the PR comment, the code reviewer wanted a full list of tasks and used ChatGPT's response to demonstrate what was wanted.[e]
Direct Link	Developers only provide the URLs to the shared conversations without any explanation.	In the issue comment, the author shared a ChatGPT conversation link without any explanation.[f]
Others	Challenges in identifying their specific purpose.	The issue comment contains content in a language other than English, although the shared conversation with ChatGPT is in English.[g]

[a] https://github.com/faker-js/faker/pull/2405#issuecomment-1723251616
[b] https://github.com/SSWConsulting/SSW.Website/pull/931#discussion_r1252442385
[c] https://github.com/rails/rails/pull/48576#issuecomment-1614443487
[d] https://github.com/GaloyMoney/blink-mobile/pull/2617#discussion_r1315920182
[e] https://github.com/OpenAdaptAI/OpenAdapt/pull/228#issuecomment-1595122865
[f] https://github.com/jabrena/spring-boot-user-beans/issues/17#issuecomment-1595563774
[g] https://github.com/scriptit-fr/ChatGPTApp/issues/3#issuecomment-1612602709

D Comparision of our Taxonomy in RQ1 with Existing Ones

Table 12 Comparison of our taxonomy in RQ1 with existing ones for Stack Overflow posts and GitHub discussion posts

Our taxonomy	Beyer et al. (2020) (Stack Overflow)	Hata et al. (2022) (GitHub discussions)
Code generation	API usage: Questions asking "how to implement something and way of using something" and "the interaction of API classes". If questions ask specifically for working code, they are categorized as "code generation" in our taxonomy.	Usage: Renamed from "API usage" proposed by Beyer et al. to emphasize general questions asking for instructions for code generation or specific functionalities.
Conceptual	Conceptual	Conceptual
How-to	API usage: Includes questions asking about "how-to". In our taxonomy,we separate questions asking for methodology from those asking for working code.	Usage: Renamed from "API usage" proposed by Beyer et al. Includes questions asking about "how-to".
Issue resolving	Discrepancy and Errors: Questions about problems and unexpected code behavior. Our "Issue resolving" contains these two types of questions and more, i.e., we consider five input types: error message, description of unexpected behaviors, issues reports, and when developers consult solutions for issues.	Discrepancy and Errors: Sourced from Beyer et al.
Review	Review	Review
Comprehension	NA	NA
Human language translation	NA	NA
Documentation	NA	NA
Information giving	NA	Information: Discussions providing general information.
Data generation	NA	NA
Data formatting	NA	NA
Math problem solving	NA	NA
Verifying capability	NA	NA
Prompt engineering	NA	NA
Execution	NA	NA
Data Analysis	NA	NA
Other	NA	Other: Discussions not fitting into other categories

Table 12 continued

Our taxonomy	Beyer et al. (2020) (Stack Overflow)	Hata et al. (2022) (GitHub discussions)
NA	API change: Questions arising from API changes or compatibility issues.	Versions: Renamed from "API change" proposed by Beyer et al. to make it more general, i.e., including different versions of the software releases or development processes.
NA	Learning: Questions asking for documentation or tutorials to learn a tool or language.	Learning: Sourced from Beyer et al.
NA	NA	Recruitment: Discussions indicating job vacancies or recruiting contributors.
NA	NA	Plans: Discussions asking about future plans of software releases or development processes.
NA	NA	Announcement: Discussions used for announcements.
NA	NA	404: Discussions that have been removed.

"NA" means the category did not have a related category in another taxonomy

Acknowledgements We acknowledge the support of the Natural Sciences and Engineering Research Council of Canada (NSERC), [funding reference number: RGPIN-2019-05071].

Data Availability The results, source code, and data related to this study are available at https://github.com/RISElabQueens/analyzing-shared-conversation

Declarations

Conflicts of Interest The authors declare that they have no conflict of interest.

References

Arya D, Wang W, Guo JL, Cheng J (2019) Analysis and detection of information types of open source software issue discussions. In: 2019 IEEE/ACM 41st international conference on software engineering (ICSE), IEEE, pp 454–464

Baltes S, Treude C, Robillard MP (2020) Contextual documentation referencing on Stack Overflow. IEEE Trans Softw Eng 48(1):135–149

Barke S, James MB, Polikarpova N (2023) Grounded copilot: how programmers interact with code-generating models. Proc ACM Program Lang 7(OOPSLA1):85–111

Barua A, Thomas S, Hassan A (2014) What are developers talking about? an analysis of topics and trends in Stack Overflow. Empir Softw Eng 19:619–654. https://doi.org/10.1007/s10664-012-9231-y

Beyer S, Macho C, Di Penta M, Pinzger M (2020) What kind of questions do developers ask on Stack Overflow? a comparison of automated approaches to classify posts into question categories. Empir Softw Eng 25:2258–2301

Chen M, Tworek J, Jun H, Yuan Q, Pinto HPdO, Kaplan J, Edwards H, Burda Y, Joseph N, Brockman G et al (2021) Evaluating large language models trained on code. arXiv:2107.03374

Deng Y, Xia CS, Yang C, Zhang SD, Yang S, Zhang L (2024) Large language models are edge-case generators: crafting unusual programs for fuzzing deep learning libraries. In: Proceedings of the 46th IEEE/ACM international conference on software engineering, pp 1–13

Di Sorbo A, Panichella S, Visaggio CA, Di Penta M, Canfora G, Gall HC (2015) Development emails content analyzer: intention mining in developer discussions (t). In: 2015 30th IEEE/ACM international conference on automated software engineering (ASE), IEEE, pp 12–23

Gagniuc PA (2017) Markov chains: from theory to implementation and experimentation. John Wiley & Sons

Gómez C, Cleary B, Singer L (2013) A study of innovation diffusion through link sharing on Stack Overflow. In: 2013 10th working conference on mining software repositories (MSR), IEEE, pp 81–84

Guo Q, Cao J, Xie X, Liu S, Li X, Chen B, Peng X (2024) Exploring the potential of chatgpt in automated code refinement: an empirical study. In: Proceedings of the 46th IEEE/ACM international conference on software engineering, pp 1–13

Hata H, Treude C, Kula RG, Ishio T (2019) 9.6 million links in source code comments: purpose, evolution, and decay. In: 2019 IEEE/ACM 41st international conference on software engineering (ICSE), IEEE, pp 1211–1221

Hata H, Novielli N, Baltes S, Kula RG, Treude C (2022) Github discussions: an exploratory study of early adoption. Empir Softw Eng 27:1–32

Hou X, Zhao Y, Liu Y, Yang Z, Wang K, Li L, Luo X, Lo D, Grundy J, Wang H (2023) Large language models for software engineering: a systematic literature review. arXiv:2308.10620

Huang Q, Xia X, Lo D, Murphy GC (2018) Automating intention mining. IEEE Trans Softw Eng 46(10):1098–1119

Jiang N, Liu K, Lutellier T, Tan L (2023) Impact of code language models on automated program repair. In: Proceedings of the 45th international conference on software engineering, IEEE Press, ICSE '23, p 1430–1442. https://doi.org/10.1109/ICSE48619.2023.00125

Landis JR, Koch GG (1977) The measurement of observer agreement for categorical data. Biometrics pp 159–174

Li L, Ren Z, Li X, Zou W, Jiang H (2018) How are issue units linked? empirical study on the linking behavior in github. In: 2018 25th Asia-pacific software engineering conference (APSEC), IEEE, pp 386–395

Liang JT, Yang C, Myers BA (2024) A large-scale survey on the usability of ai programming assistants: Successes and challenges. In: Proceedings of the 46th IEEE/ACM international conference on software engineering, pp 1–13

Liu J, Xia X, Lo D, Zhang H, Zou Y, Hassan AE, Li S (2021) Broken external links on Stack Overflow. IEEE Trans Softw Eng 48(9):3242–3267

Liu J, Zhang H, Xia X, Lo D, Zou Y, Hassan AE, Li S (2022) An exploratory study on the repeatedly shared external links on Stack Overflow. Empir Softw Eng 27:1–32

Lu J, Yu L, Li X, Yang L, Zuo C (2023) Llama-reviewer: advancing code review automation with large language models through parameter-efficient fine-tuning. In: 2023 IEEE 34th international symposium on software reliability engineering (ISSRE), IEEE, pp 647–658

Mozannar H, Bansal G, Fourney A, Horvitz E (2022) Reading between the lines: modeling user behavior and costs in ai-assisted programming. arXiv:2210.14306

Nurwidyantoro A, Shahin M, Chaudron MR, Hussain W, Shams R, Perera H, Oliver G, Whittle J (2022) Human values in software development artefacts: a case study on issue discussions in three Android applications. Inf Softw Technol 141:106731

OpenAI (2024a) ChatGTP Shared Links FAQ. https://help.openai.com/en/articles/7925741-chatgpt-shared-links-faq. Accessed 23 Jan 2024

OpenAI (2024b) Create a Shared Link. https://help.openai.com/en/articles/7943611-create-a-shared-link. Accessed 23 Jan 2024

Qu C, Yang L, Croft WB, Zhang Y, Trippas JR, Qiu M (2019) User intent prediction in information-seeking conversations. In: Proceedings of the 2019 conference on human information interaction and retrieval, pp 25–33

Rosen C, Shihab E (2016) What are mobile developers asking about? a large scale study using Stack Overflow. Empir Softw Eng 21:1192–1223

Ross SI, Martinez F, Houde S, Muller M, Weisz JD (2023) The programmer's assistant: conversational interaction with a large language model for software development. In: Proceedings of the 28th international conference on intelligent user interfaces, pp 491–514

Shi L, Chen X, Yang Y, Jiang H, Jiang Z, Niu N, Wang Q (2021) A first look at developers' live chat on gitter. In: Proceedings of the 29th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering, pp 391–403

Siddiq ML, Santos J, Tanvir RH, Ulfat N, Rifat FA, Lopes VC (2023) Exploring the effectiveness of large language models in generating unit tests. arXiv:2305.00418

Treude C, Barzilay O, Storey MA (2011) How do programmers ask and answer questions on the web?(nier track). In: Proceedings of the 33rd international conference on software engineering, pp 804–807

Vaithilingam P, Zhang T, Glassman EL (2022) Expectation vs. experience: evaluating the usability of code generation tools powered by large language models. In: Chi conference on human factors in computing systems extended abstracts, pp 1–7

Viviani G, Famelis M, Xia X, Janik-Jones C, Murphy GC (2019) Locating latent design information in developer discussions: a study on pull requests. IEEE Trans Softw Eng 47(7):1402–1413

Wan Z, Xia X, Hassan AE (2019) What is discussed about blockchain? a case study on the use of balanced lda and the reference architecture of a domain to capture online discussions about blockchain platforms across the stack exchange communities. IEEE Trans Softw Eng 01:1–1

Wang D, Xiao T, Thongtanunam P, Kula RG, Matsumoto K (2021) Understanding shared links and their intentions to meet information needs in modern code review: a case study of the openstack and qt projects. Empir Softw Eng 26:1–32

Xiao T, Baltes S, Hata H, Treude C, Kula RG, Ishio T, Matsumoto K (2023) 18 million links in commit messages: purpose, evolution, and decay. Empir Softw Eng 28(4):91

Xiao T, Treude C, Hata H, Matsumoto K (2024) Devgpt: studying developer-chatgpt conversations. In: Proceedings of the International Conference on Mining Software Repositories (MSR 2024)

Ye D, Xing Z, Kapre N (2017) The structure and dynamics of knowledge network in domain-specific q&a sites: a case study of Stack Overflow. Empir Softw Eng 22:375–406

Zampetti F, Ponzanelli L, Bavota G, Mocci A, Di Penta M, Lanza M (2017) How developers document pull requests with external references. In: 2017 IEEE/ACM 25th International conference on program comprehension (ICPC), IEEE, pp 23–33

Zhang B, Liang P, Zhou X, Ahmad A, Waseem M (2023) Practices and challenges of using github copilot: an empirical study. arXiv:2303.08733

Zhang T, Gao C, Ma L, Lyu M, Kim M (2019) An empirical study of common challenges in developing deep learning applications. In: 2019 IEEE 30th international symposium on software reliability engineering (ISSRE), IEEE, pp 104–115

Zhang Y, Yu Y, Wang H, Vasilescu B, Filkov V (2018) Within-ecosystem issue linking: a large-scale study of rails. In: Proceedings of the 7th international workshop on software mining, pp 12–19

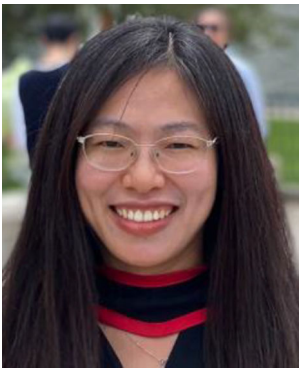
Ziegler A, Kalliamvakou E, Li XA, Rice A, Rifkin D, Simister S, Sittampalam G, Aftandilian E (2022) Productivity assessment of neural code completion. In: Proceedings of the 6th ACM SIGPLAN international symposium on machine programming, pp 21–29

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Huizi Hao is a research-based Master's student at the School of Computing at Queen's University, Canada. Prior to her Master's study, she worked as a software developer. She earned her Bachelor of Computer Science Honours degree from the University of Manitoba. Her research interests lie in Software Engineering and Artificial Intelligence, which include empirical software engineering, data mining, Software Engineering for AI (SE4AI) and AI for Software Engineering (AI4SE).

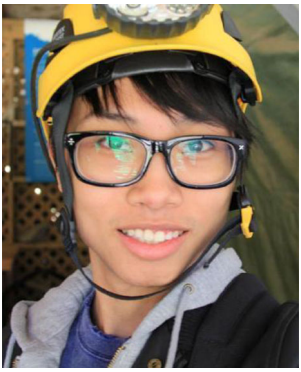
Kazi Amit Hasan is a PhD student at the School of Computing, Queen's University. Prior to his doctoral studies, he worked as a Machine Learning Engineer at ACI Limited in Bangladesh. He earned his Bachelor of Science in Computer Science & Engineering from Rajshahi University of Engineering & Technology (RUET), Bangladesh. His research interests include, but are not limited to data mining, mining software repository, empirical software engineering and intelligent pull request management. His professional experience includes a Mitacs Business Strategy Internship Award, contributions to open-source projects, and achieving top ranks in competitive machine learning competitions. More details can be found at: https://amithasanshuvo.github.io/.



Hong Qin is an Analysis Programmer in the School of Medicine at Duke University. Hong earned her Master's degree from Queen's University in 2024. Prior to her Master's study, she worked at the Department of Mechanical and Material Engineering of Queen's University as a postdoctoral research fellow. Her current research focuses on magnetic resonance imaging (MRI) research with hyperpolarized 129Xe gas, developing this novel technology for imaging the lungs of patients with pulmonary disease.

Marcos Macedo is a PhD student at the School of Computing, Queen's University. He holds a BSc in Computer Science from the University of Montevideo, Uruguay. His current research focuses on Large Language Models for Software Engineering (AI4SE) and Software Engineering for AI (SE4AI), particularly in code translation across programming languages, improving benchmarking methodologies and simplifying LLM data pipelines to make them more accessible to users with diverse technical backgrounds. Before Queen's, Marcos worked at NASDAQ Verafin and IBM, with over 10 years of experience in software development, advocacy, consulting, and mentoring.

Yuan Tian is an Assistant Professor in the School of Computing at Queen's University, Canada. She earned her Ph.D. from Singapore Management University in 2017. Yuan leads the RISE research lab, focusing on automating software development tasks, including software bug and technical debt management, automated code transformation, pull request management, and software engineering for AI. Her work has been featured in top SE conferences and journals, and she has served on program and organization committees for key SE conferences.



Steven H.H. Ding is an Assistant Professor in the School of Information Studies at McGill University, building on his previous role as an Assistant Professor in the School of Computing at Queen's University. His research focuses on harnessing AI technologies to address the complex challenges of cybersecurity while enhancing the resilience of AI systems for the future. Dr. Ding earned his Ph.D. from McGill University in 2019, during which he received the FRQNT Doctoral Research Scholarship of Quebec and the Dean's Graduate Award at McGill. His current research is supported by BlackBerry, the Canadian Centre for Cyber Security (CCCS), and Defence Research and Development Canada (DRDC).

Ahmed E. Hassan is an ACM Fellow, IEEE fellow, and an NSERC Steacie Fellow. He is the Canada Research Chair (CRC) in Software Analytics, and the NSERC/BlackBerry Software Engineering Chair at the School of Computing at Queen's University, Canada. He is a Distinguished Educator of IEEE TCSE and an Influential Educator of ACM SIGSOFT. He is a recipient of the 2020 inaugural New Directions IEEE TCSE award for outstanding and sustained contributions to establishing a new field of research, the Mining Software Repositories field, which examines the application of machine learning to Software Engineering data. His research interests include mining software repositories, empirical software engineering, load testing, and log mining. He received a PhD in Computer Science from the University of Waterloo. He also serves/d on the editorial boards of IEEE Transactions on Software Engineering, Springer Journal of Empirical Software Engineering, and PeerJ Computer Science. Contact ahmed@cs.queensu.ca. More information at: http://sail.cs.queensu.ca/.

Authors and Affiliations

Huizi Hao[1] · Kazi Amit Hasan[1] · Hong Qin[1] · Marcos Macedo[1] · Yuan Tian[1] · Steven H.H. Ding[1] · Ahmed E. Hassan[2]

✉ Yuan Tian
y.tian@queensu.ca

Huizi Hao
huizi.hao@queensu.ca

Kazi Amit Hasan
kaziamit.hasan@queensu.ca

Hong Qin
21hq5@queensu.ca

Marcos Macedo
marcos.macedo@queensu.ca

Steven H.H. Ding
steven.ding@queensu.ca

Ahmed E. Hassan
ahmed@cs.queensu.ca

1 School of Computing, Queen's University, ON Kingston, Canada

2 Software Analysis and Intelligence Lab (SAIL), ON Kingston, Canada

Reproduced with permission of copyright owner.
Further reproduction prohibited without permission.